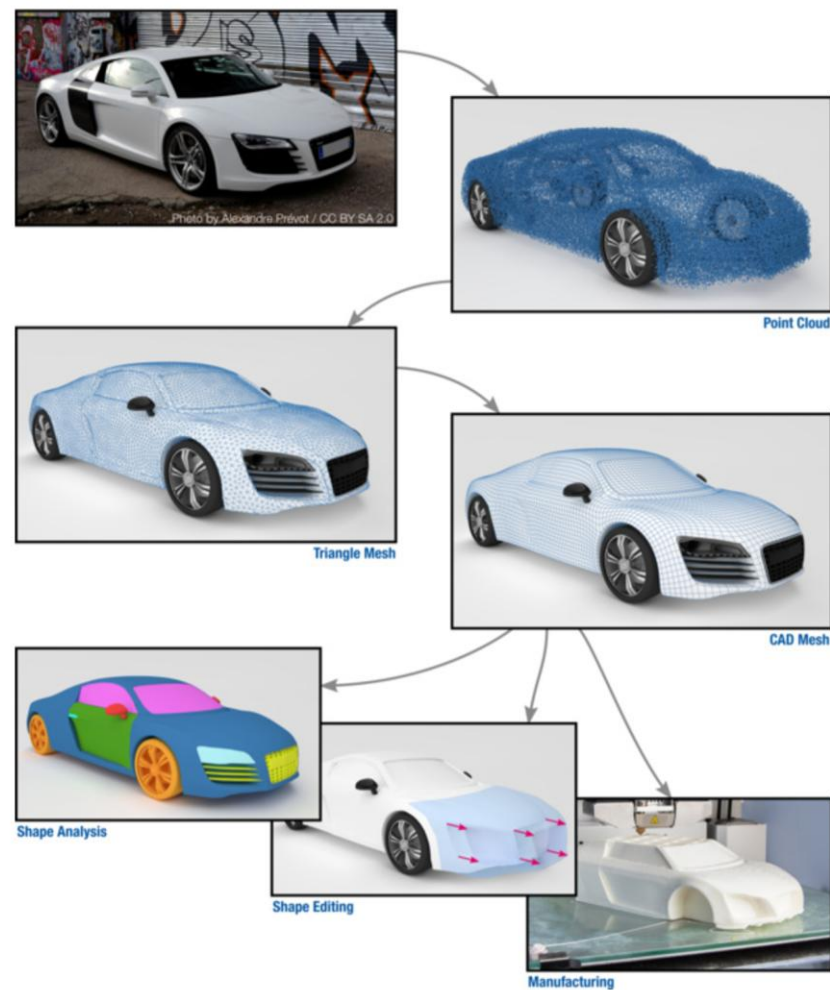


数字几何处理

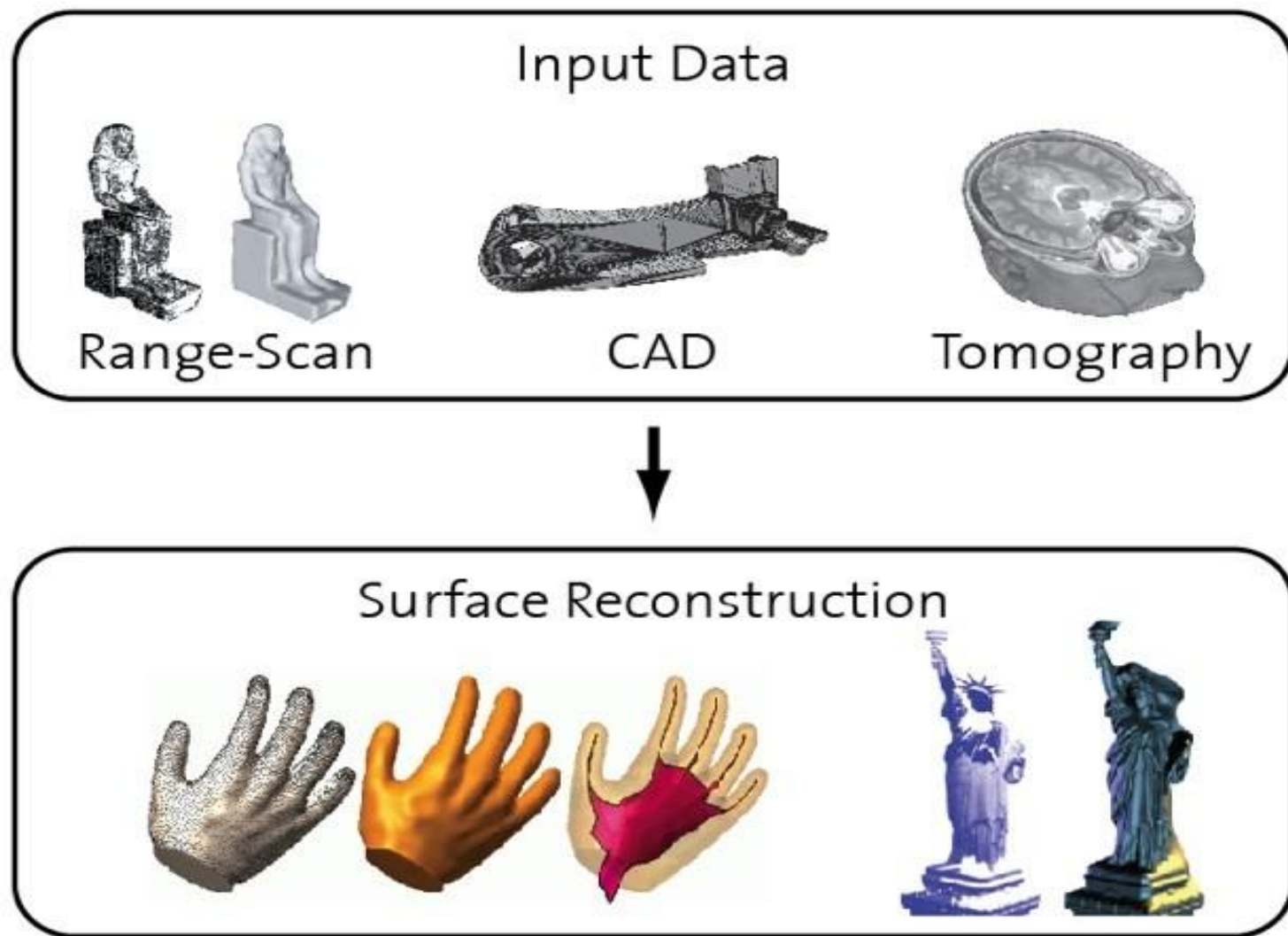
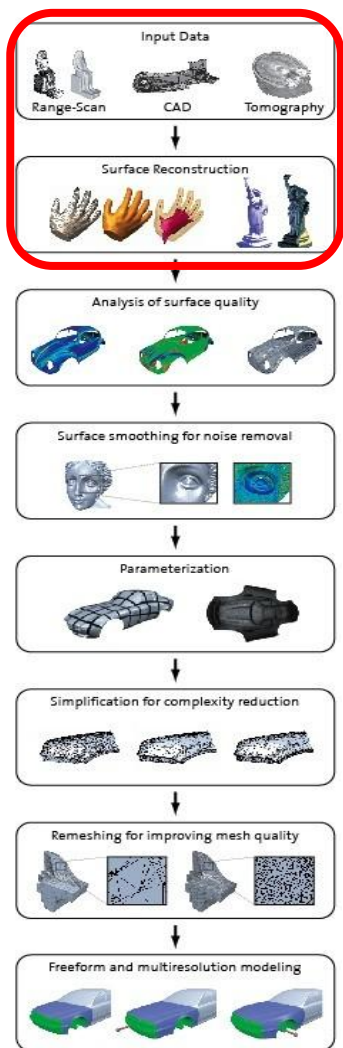
高林

中国科学院计算技术研究所

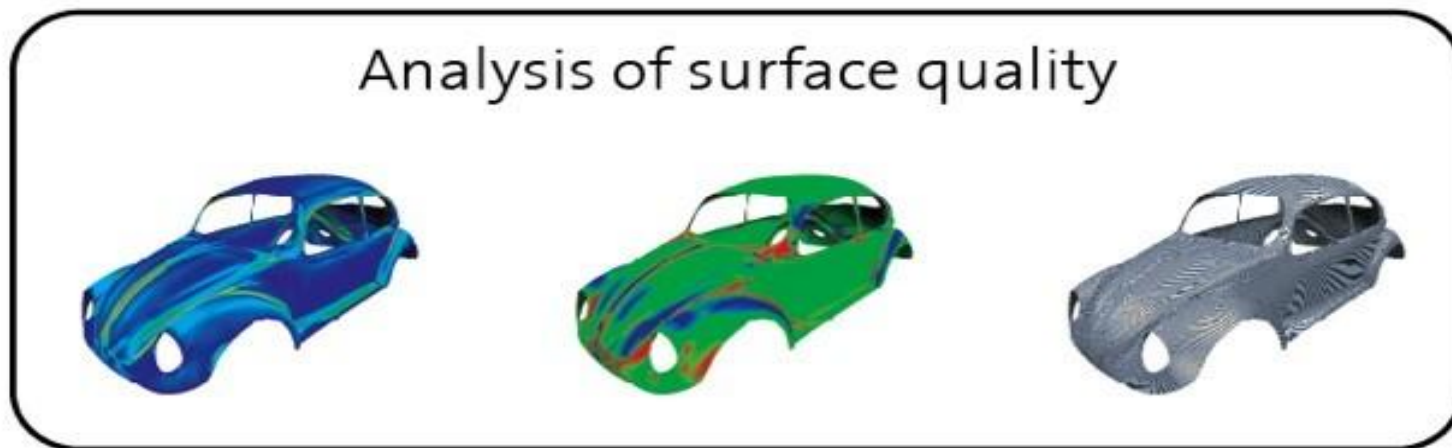
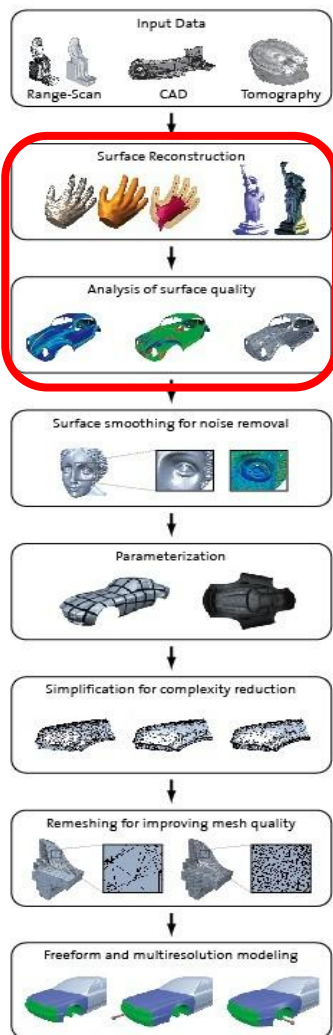
几何处理流程



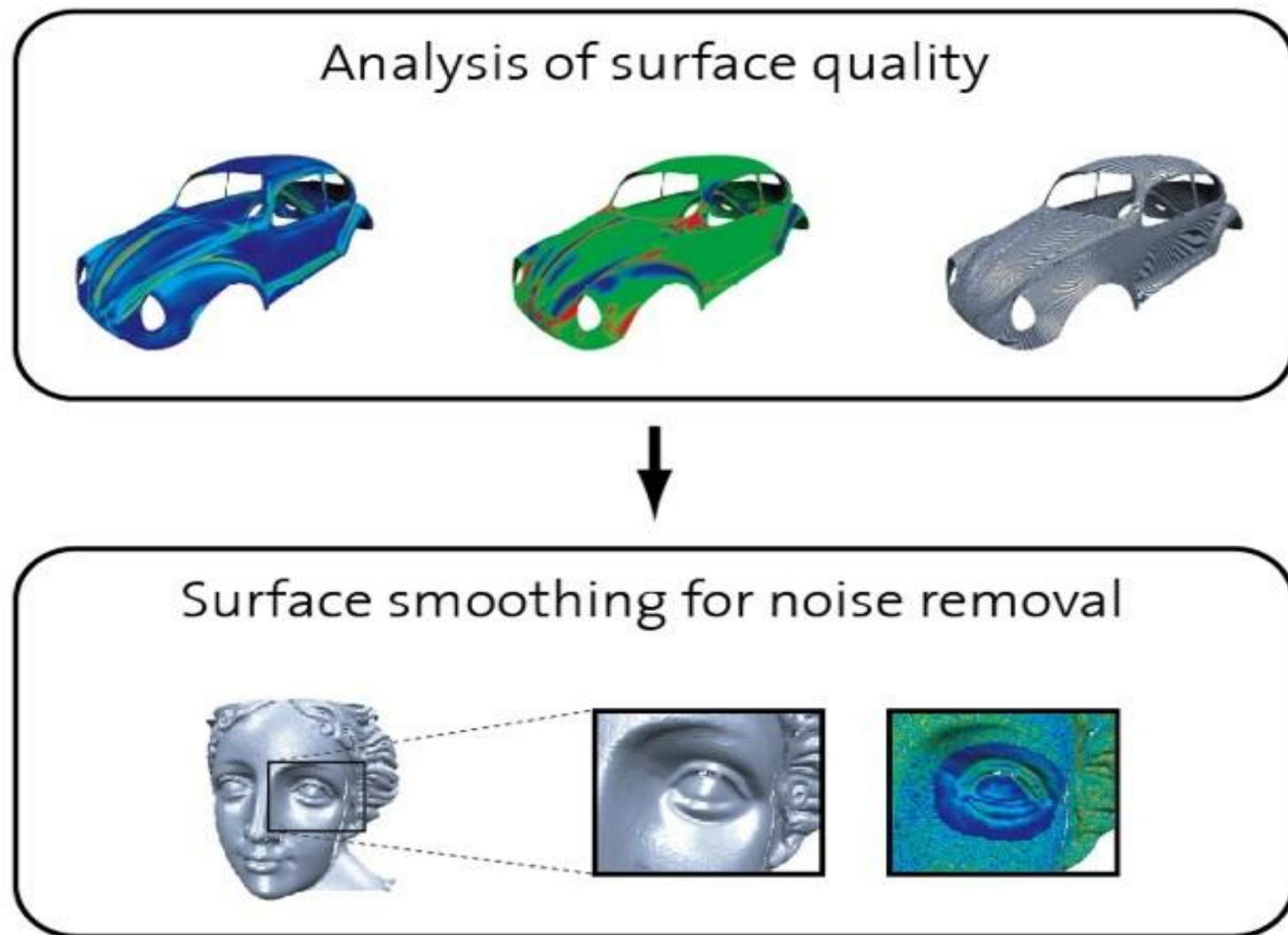
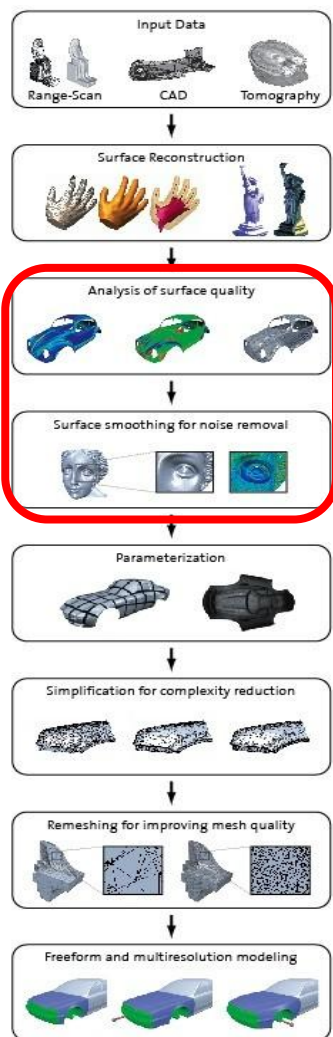
几何处理流程



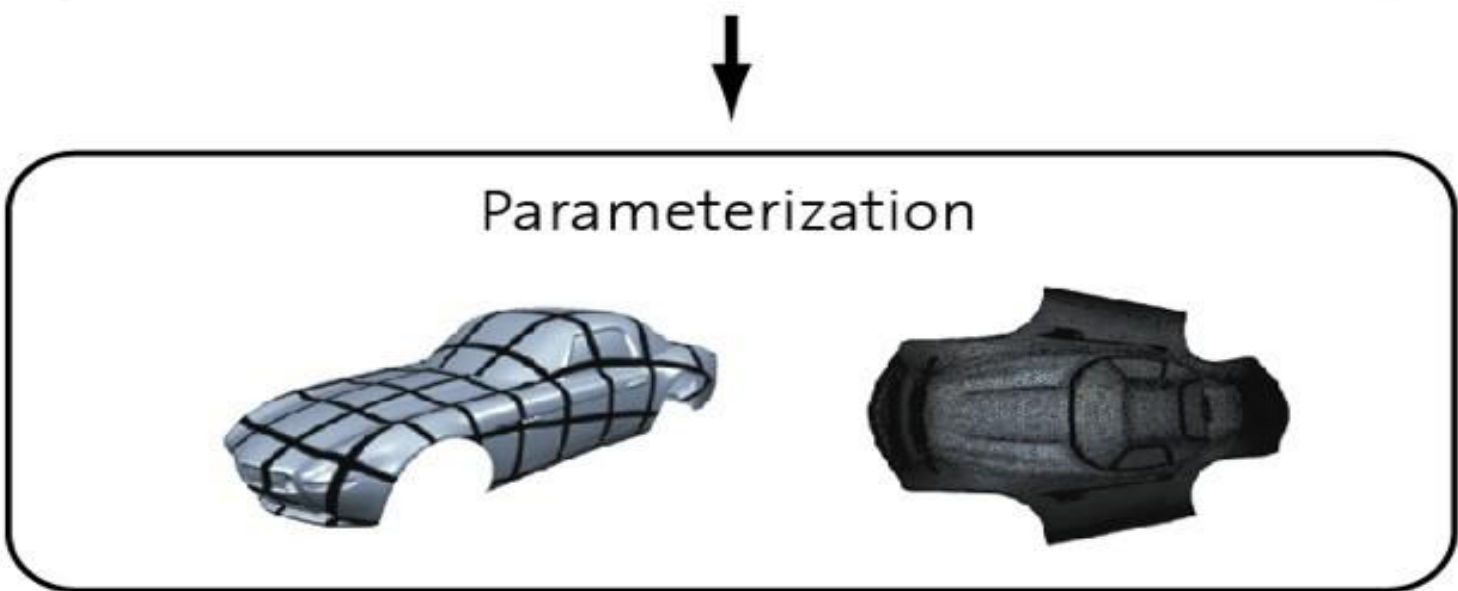
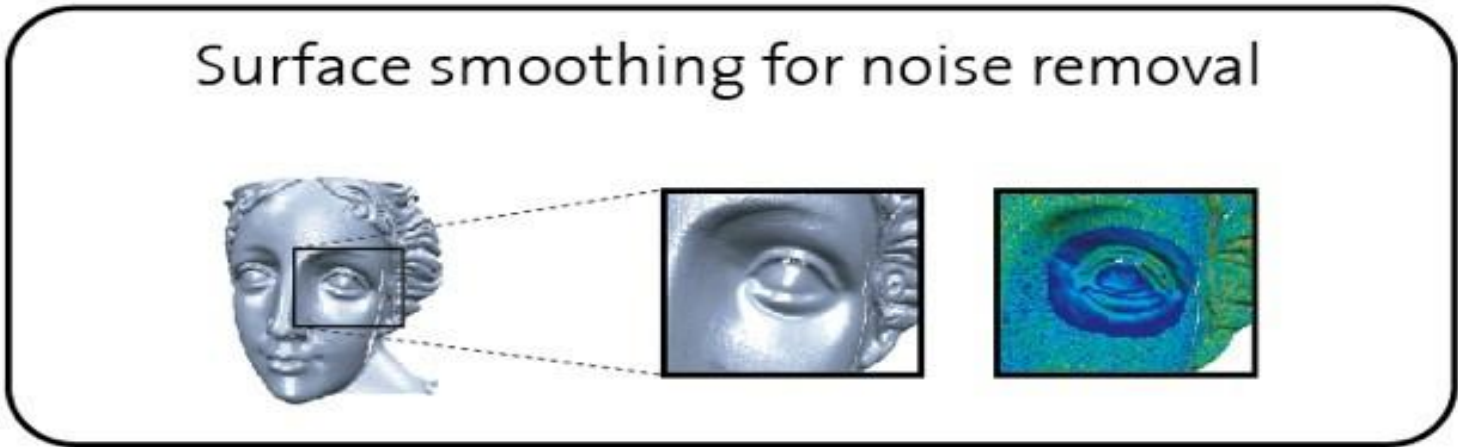
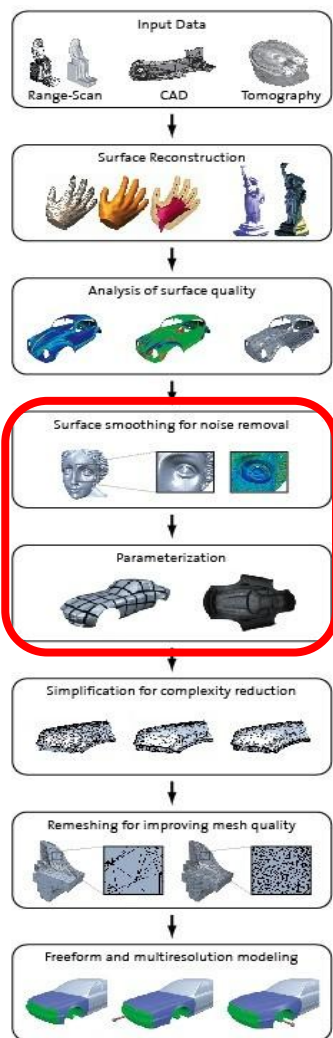
几何处理流程



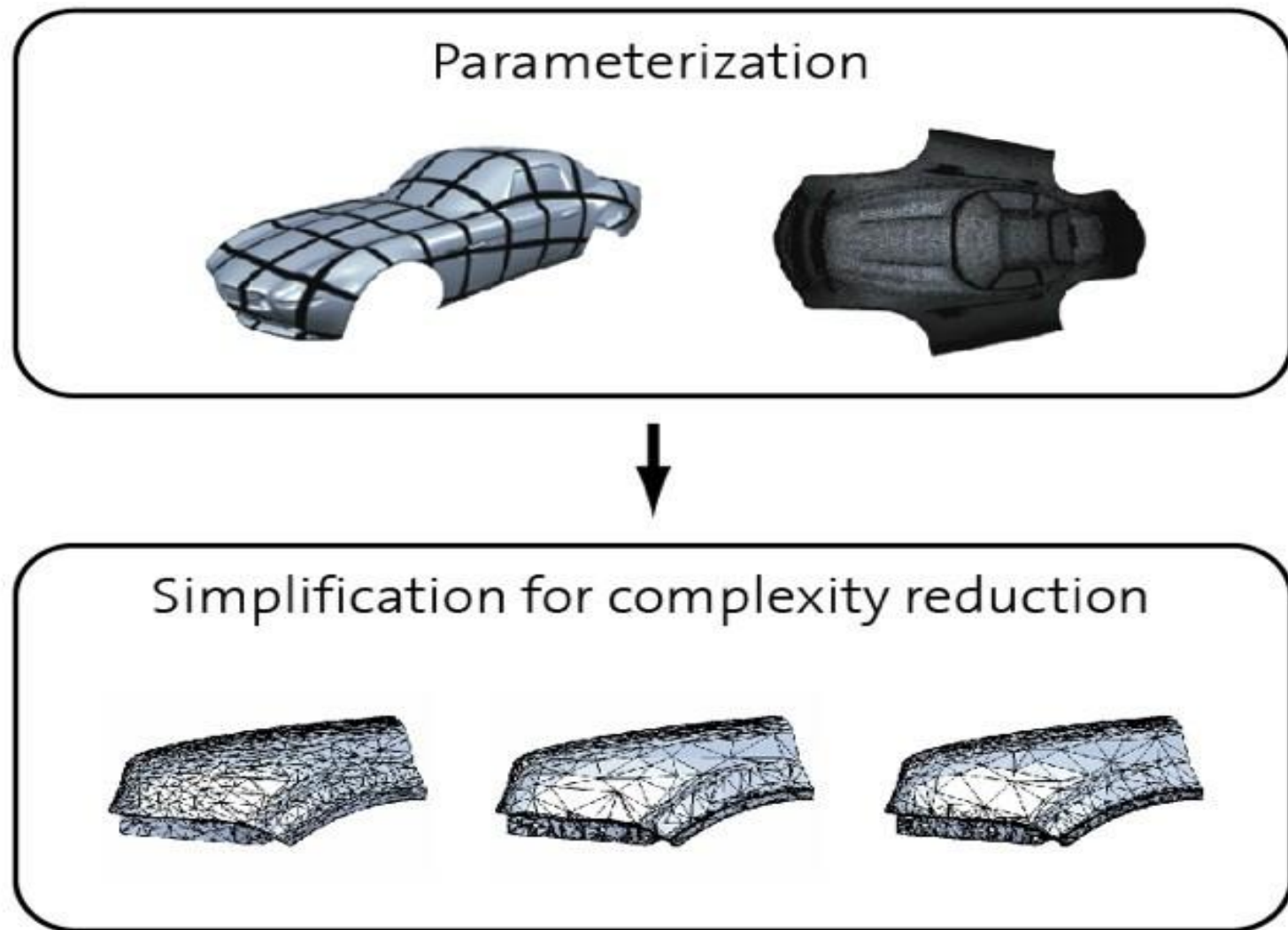
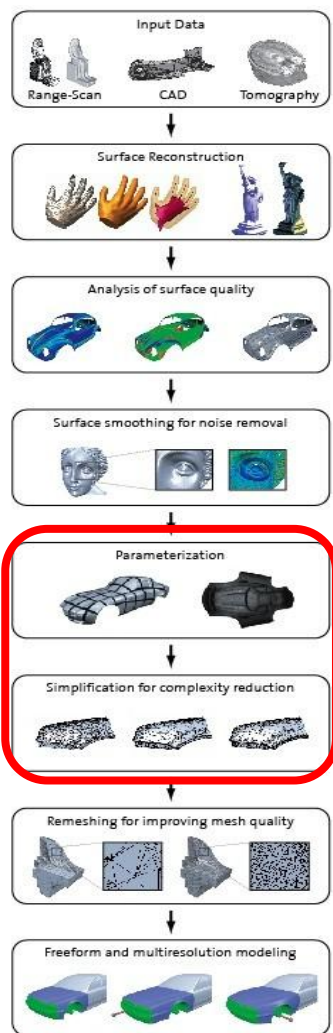
几何处理流程



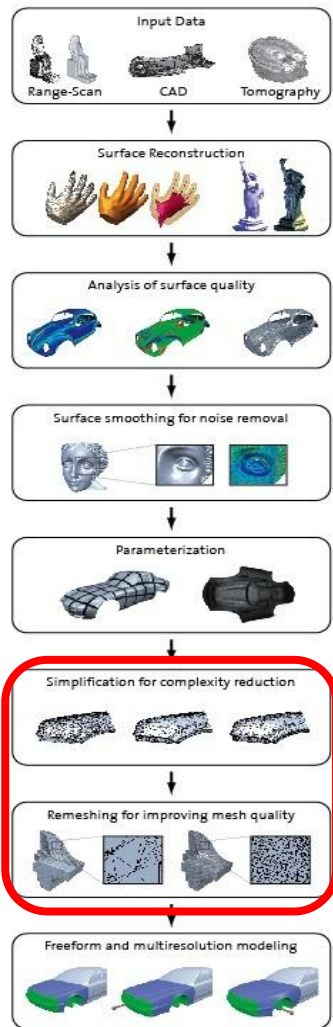
几何处理流程



几何处理流程



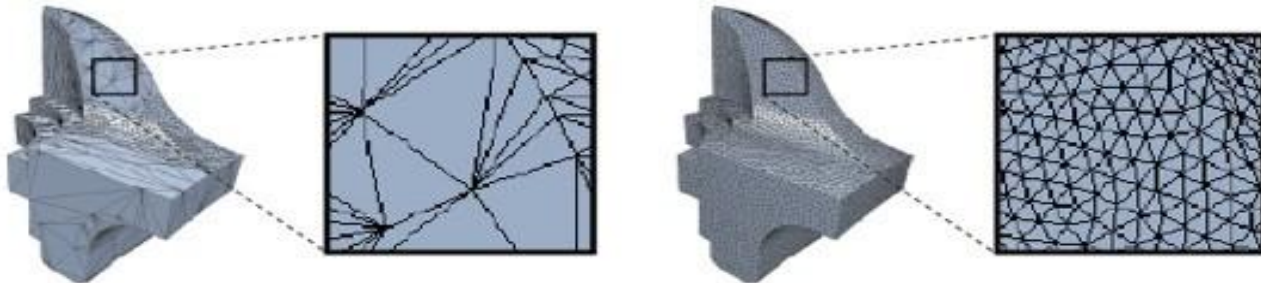
几何处理流程



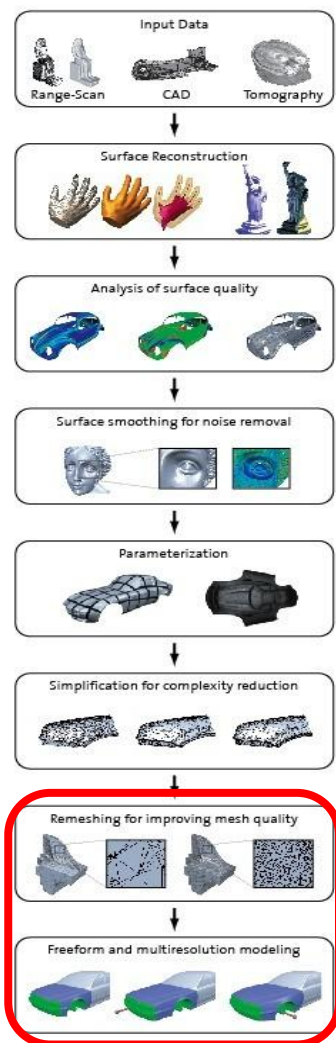
Simplification for complexity reduction



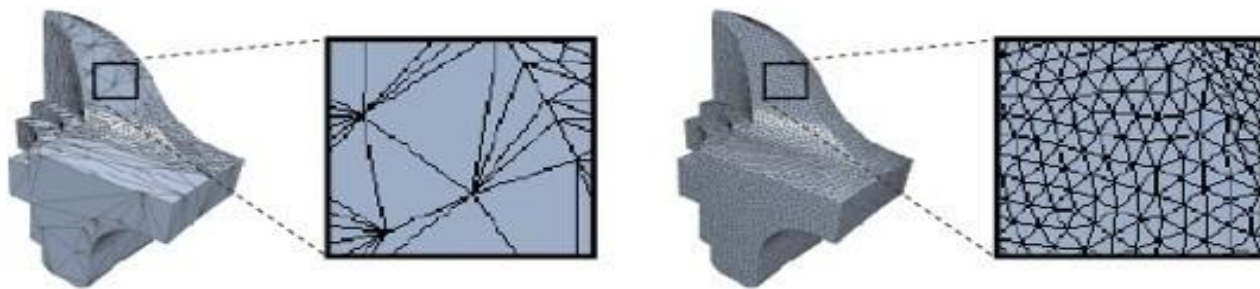
Remeshing for improving mesh quality



几何处理流程



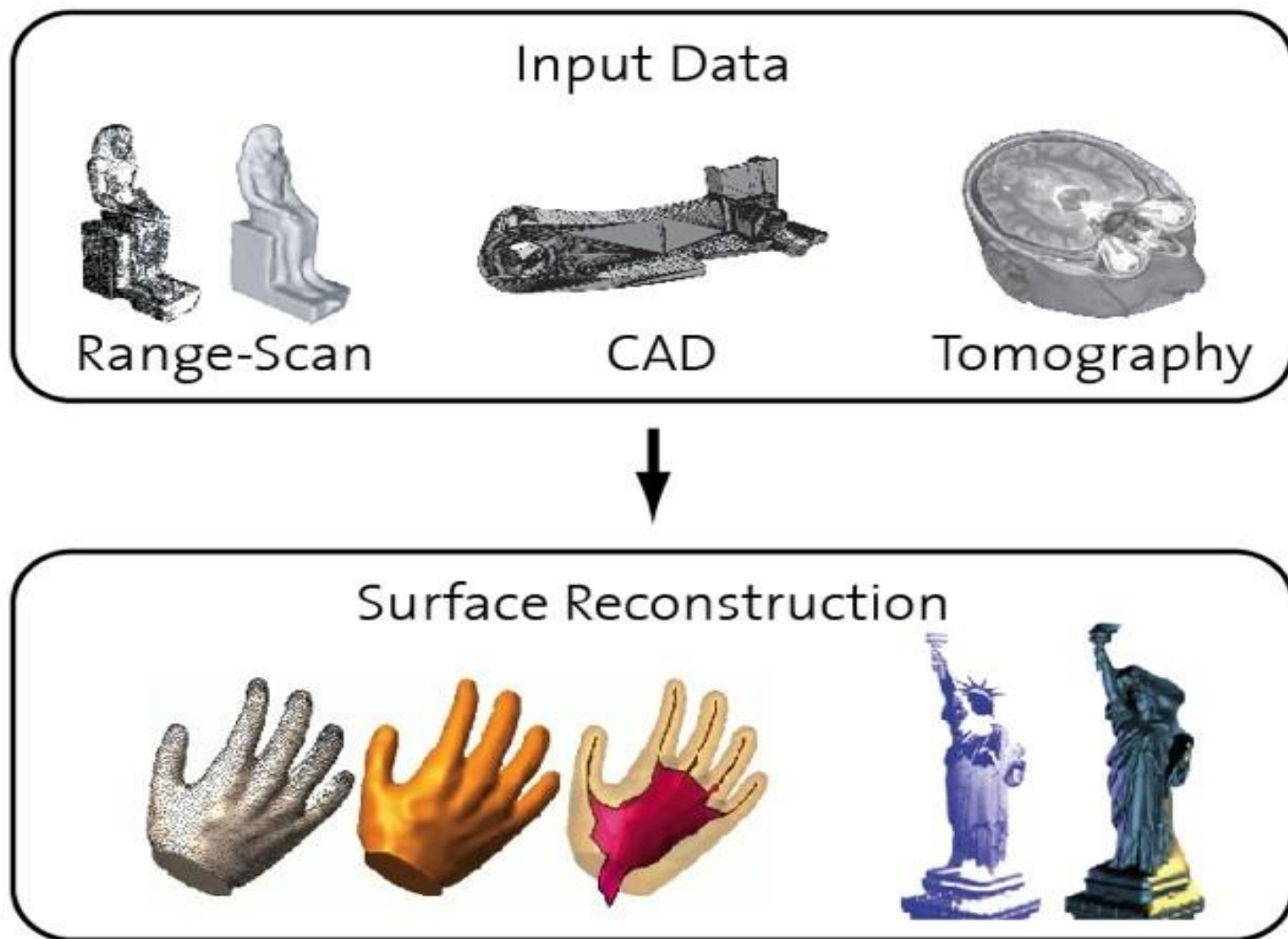
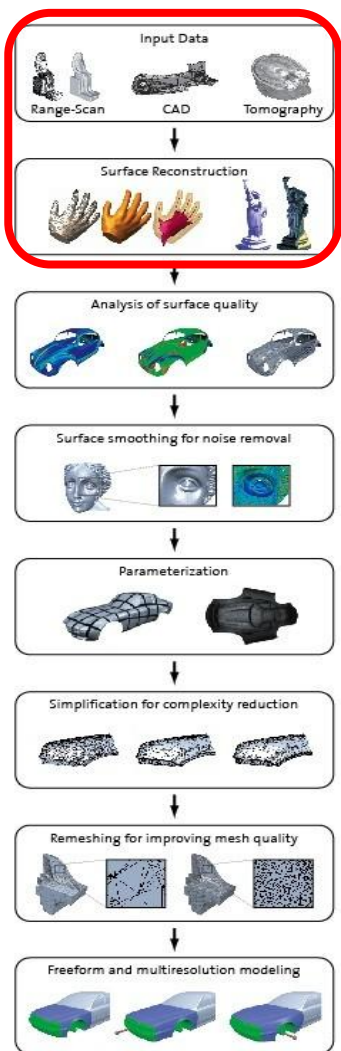
Remeshing for improving mesh quality



Freeform and multiresolution modeling



几何处理流程



输入数据

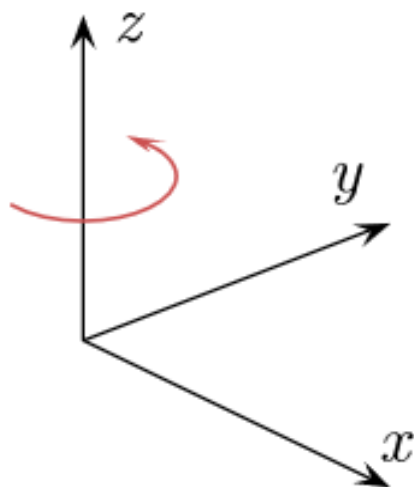
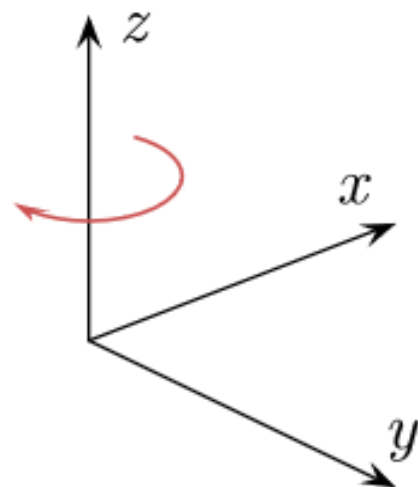
■ 几何建模

- ◆ 实体模型三维扫描
- ◆ CAD 人机交互
- ◆ CT/ MRI
- ◆ 计算机模拟

坐标系

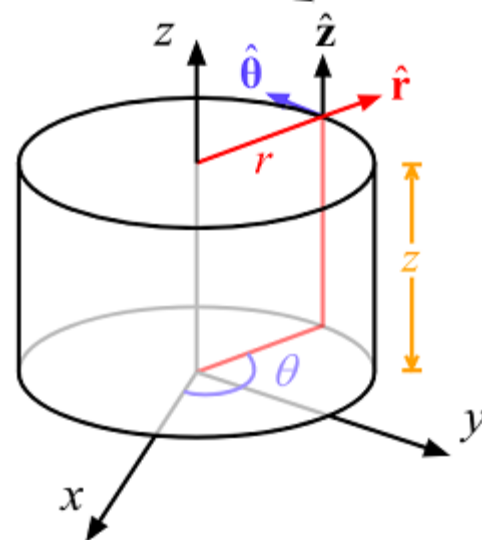
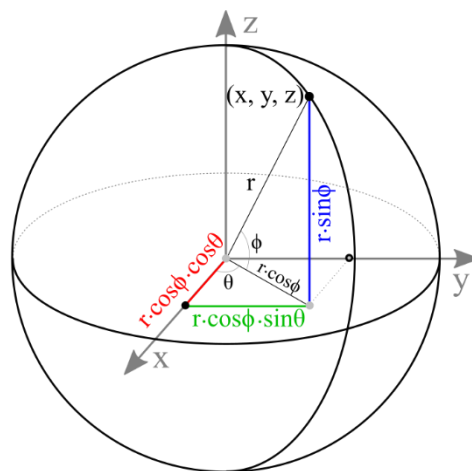
■ 直角坐标系(x, y, z)

◆ 左手系、右手系



■ 球坐标系(r, θ, ϕ)

■ 柱坐标系(r, h, θ)



世界坐标 vs 局部坐标

■ 局部坐标系优势

- ◆ 表示简洁
- ◆ 同一个物体多次出现时，可以通过变换得到新的物体
 - 标准体素+变换=新的物体
- ◆ 便于进行几何操作

■ 世界坐标系和局部坐标系之间相互转换

- ◆ 单的线性变换，如平移、旋转、放缩、剪切以及这些简单变换的组合
- ◆ 变换的组合由齐次坐标表示下的变化矩阵的乘积定义

三维模型数据结构

■ 点云

- ◆ 存储 (x, y, z) 坐标、颜色、时间等信息
- ◆ 三维激光扫描、二维影像三维重建、三维模型逆向点云等



■ 体素

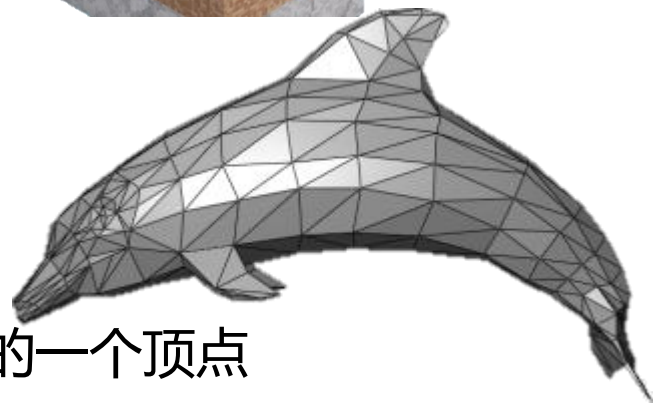
- ◆ 如同像素，是数字数据于三维空间分割上的最小单位



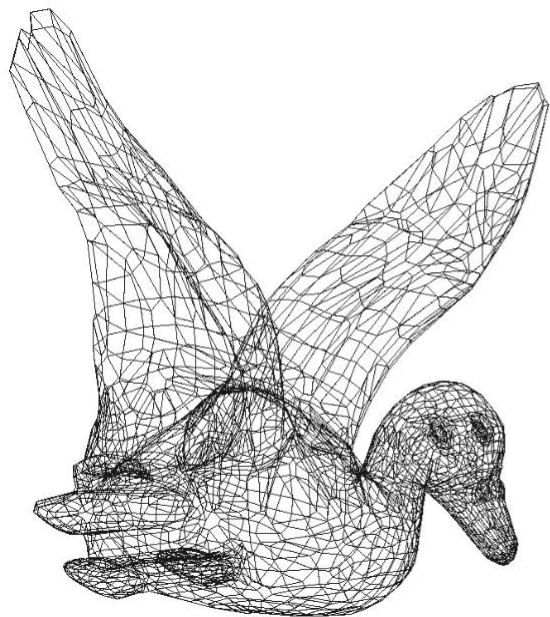
■ 网格

◆ 多边形网格模型

- 多边形称为网格的面，多边形的顶点也称为网格的顶点
- 一般要求两张相邻面的公共边完全相同，即不能出现某一面的一个顶点在另一面的边中间



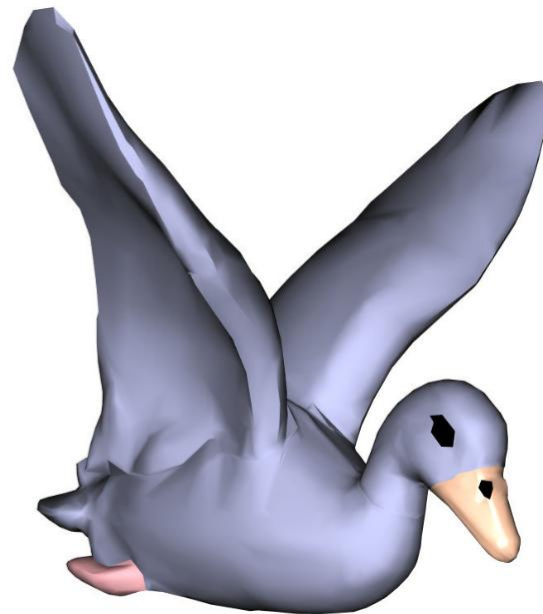
多边形表示



线框图



直接用光照明
模型进行绘制



Phong
明暗处理

多边形表示

- 大量的平面片表示（三角形、四边形等）
 - ◆ 三角形最为常见
 - ◆ 本质：线性表示形式
- 多边形应当封闭
- 用封闭的多边形面片表示三维物体
 - ◆ 顶点个数与面片的个数存在确定的关系

常见数据格式

■ OBJ格式

- ◆ wavefront 公司开发
- ◆ 动画文件描述
- ◆ 几何、颜色、纹理、材质等多种属性
- ◆ 支持曲线曲面
- ◆ 顶点坐标表 $v_i = (x_i, y_i, z_i)$ 、纹理坐标表 $vt_p = (u_p, v_p)$ 、法向表 $vn_a = (n_{xa}, n_{ya}, n_{za})$ 、面表 $f_s = (v_i/vt_p/vn_a, v_j/vt_q/vn_b, \dots)$

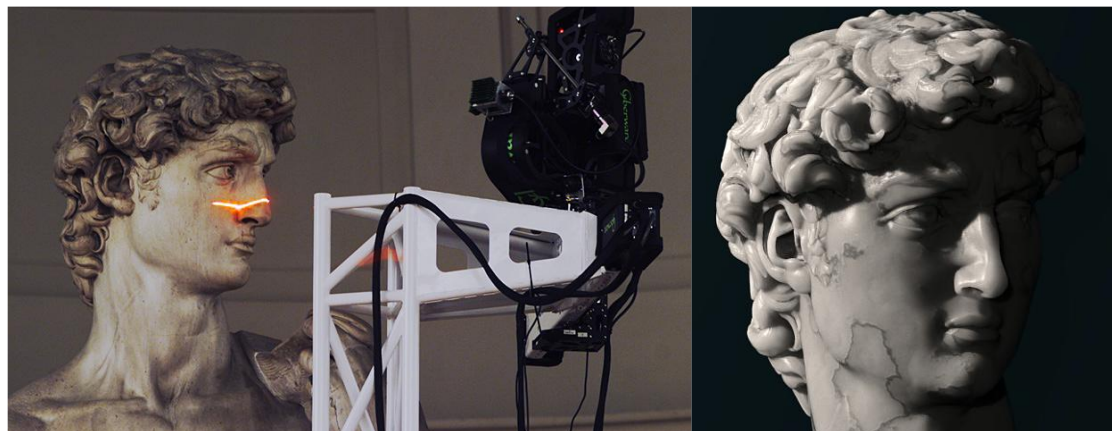
其它常见数据格式

■ STL

- ◆ STereoLithography, 光固化立体造型术
- ◆ 定义几何, 用于3D 打印

■ PLY

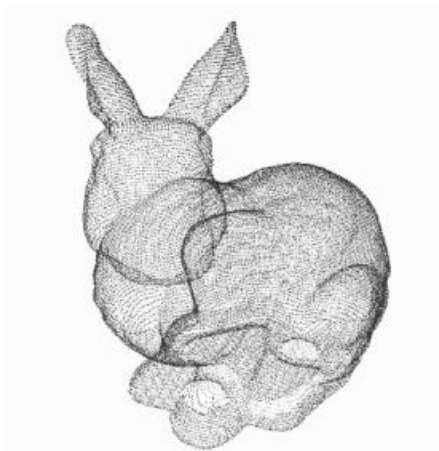
- ◆ 结构简单, 以ASCII码格式或二进制形式存储文件
- ◆ 斯坦福大学的数字米开朗琪罗项目
- ◆ 多用于3D 扫描



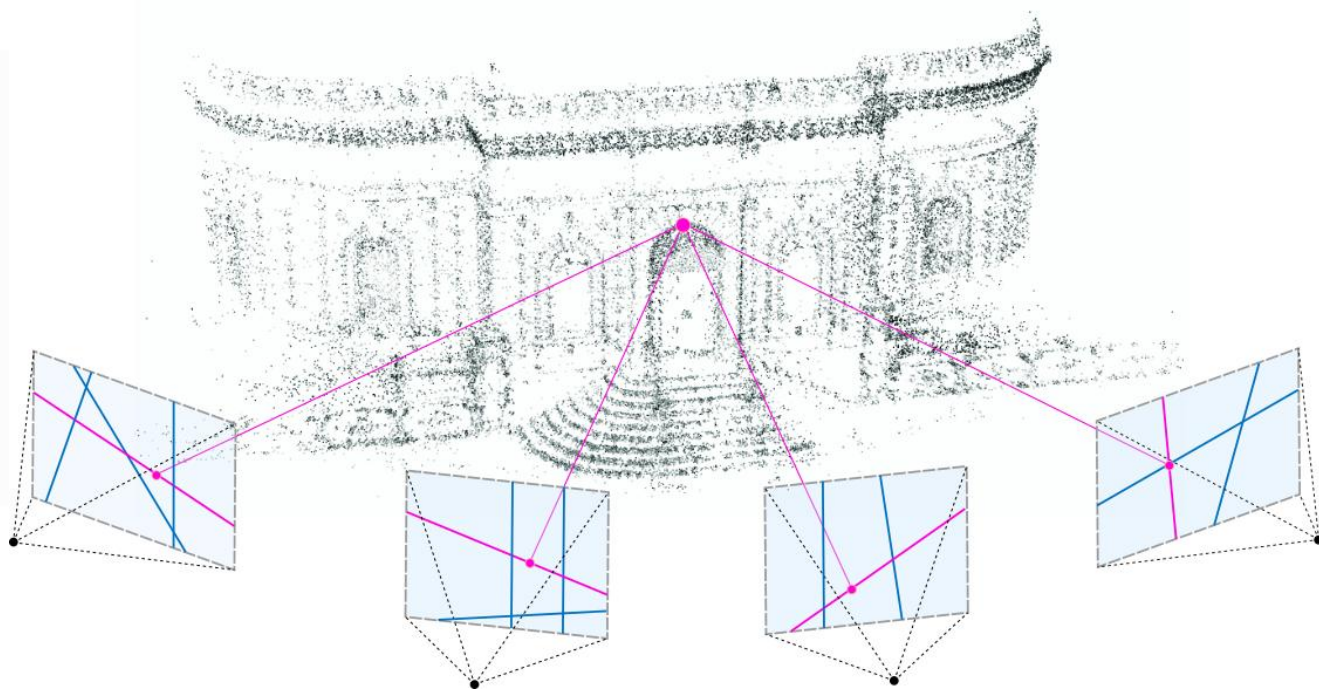
点云的获取

■ 点云的获取

- ◆ 直接采集
- ◆ 从多视角2D图片重建点云

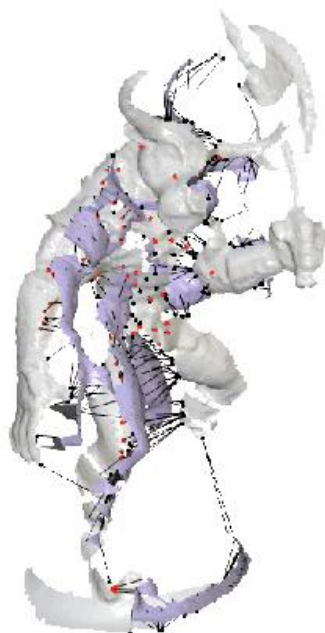


点云



图像配准

根据相关变换的初始猜测
对齐两个部分重叠的网格
结构

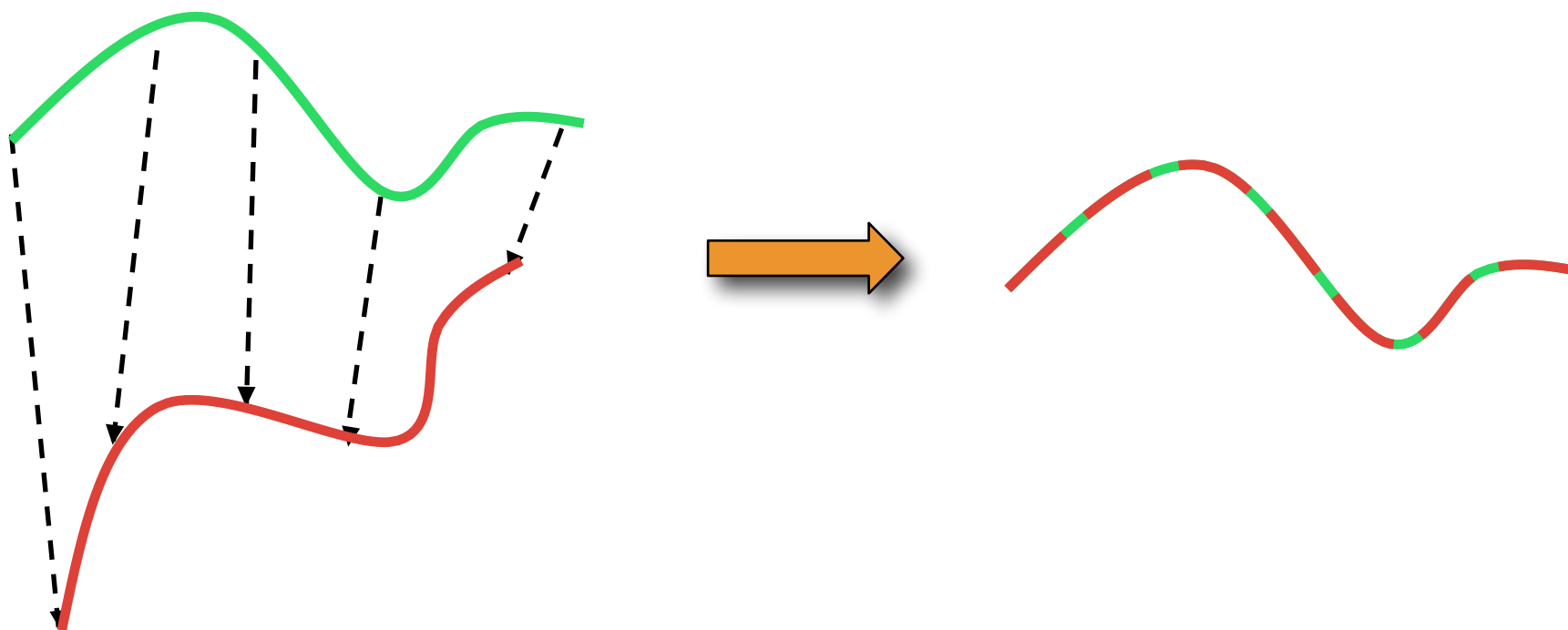


内容

- ICP: 迭代最近点 (Iterative Closest Points)
- ICP变体分类
 - 更快对齐
 - 更具鲁棒性
- ICP作为函数进行最小化

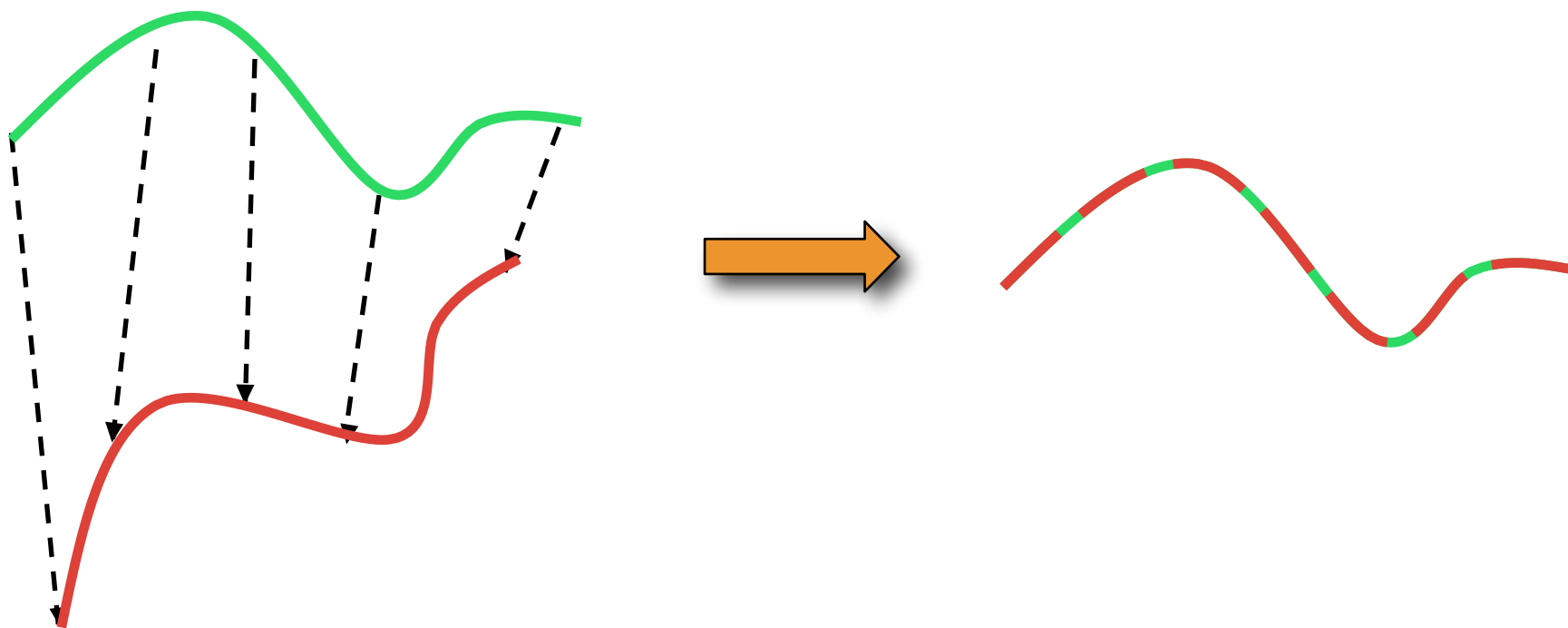
对齐3D数据

如果正确的对应关系已知，如何够找到正确的旋转/变换？



对齐3D数据

- 如何找到对应关系：用户输入？ 特征识别？ 标志？
- 替代方案：假设**最近点**相一致

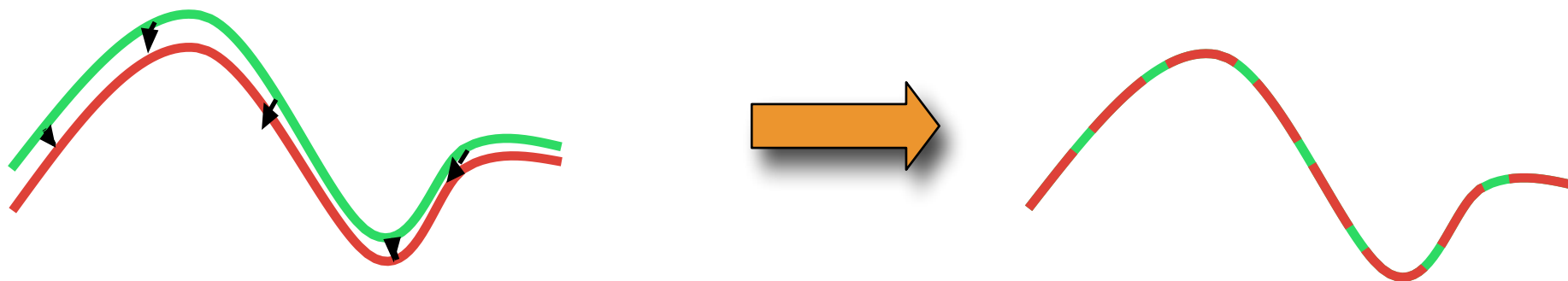


对齐3D数据

- ...如何找到对齐方式?

- ◆ Iterative Closest Points (ICP) [Besl & Mckay, 1992]

- 如果起始位置 “足够接近” 则收敛(Converges if starting position “close enough”)



基础的ICP

- **源点云P**， **目标点云Q**
- 在**源点云P**中采样若干个**随机点 p_i** (如100个)
- 对于每个**随机点 p_i** ， 在**目标点云Q**中找到距离该**随机点 p_i** 的**最近点 q_i** ， 可使用例如k-d树进行临近查找
- **拒绝**距离 大于 k 倍中位数的点对， 构建损失函数
 - ◆ $E = \sum \|Rp_i + t - q_i\|^2$
 - ◆ 其中 q_i 、 p_i 点对， R 和 t 为旋转矩阵和平移向量
- **最小化**损失函数(closed form solution in [Horn 87])

图形匹配：平移

- 定义重心点集 (bary-centered point sets)

- ◆ $\bar{p} := \frac{1}{m} \sum_{i=1}^m p_i$ $\bar{q} := \frac{1}{m} \sum_{i=1}^m q_i$
 $\hat{p}_i := p_i - \bar{p}$ $\hat{q}_i := q_i - \bar{q}$

- 优化变换向量 (translation vector) t 将中心点映射到另一个模型

- ◆ $t = \bar{p} - R\bar{q}$

图形匹配：旋转

- 通过通用矩阵（general matrix）估计非线性旋转

$$\diamond \min_R \sum_i \|\hat{p}_i - R\hat{q}_i\|^2 \quad \rightarrow \quad \min_A \sum_i \|\hat{p}_i - A\hat{q}_i\|^2$$

- 最小二乘变换矩阵为

$$\diamond A = \left(\sum_{i=1}^m \hat{p}_i \hat{q}_i^T \right) \cdot \left(\sum_{i=1}^m \hat{q}_i \hat{q}_i^T \right)^{-1} \in \mathbb{R}^{3 \times 3}$$

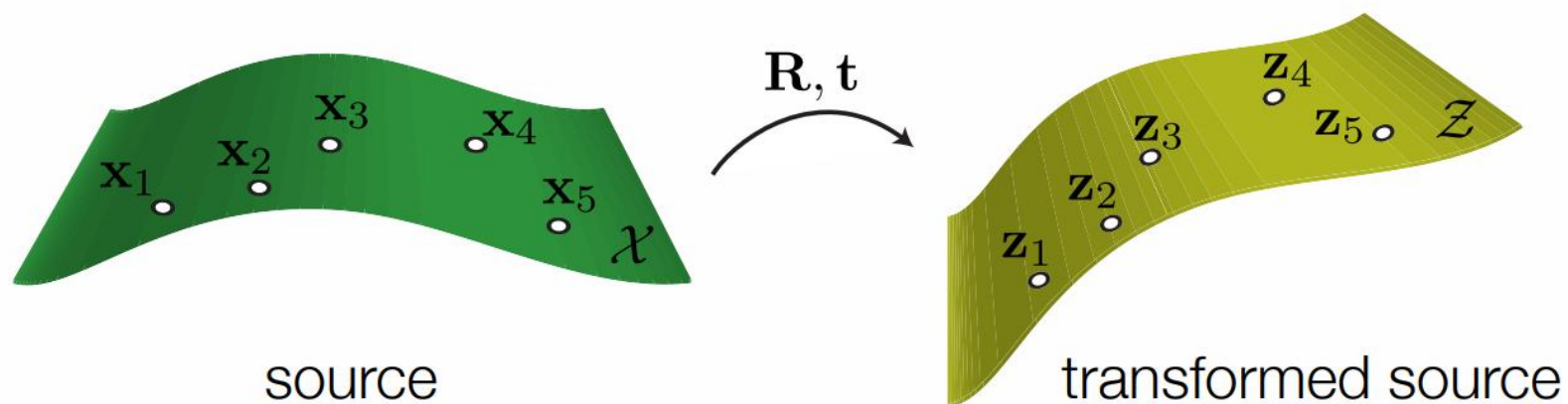
- SVD & Polar 分解：根据 A 求解旋转矩阵

$$\diamond A = U\Sigma V^T \quad \rightarrow \quad R = UV^T$$

3D配准 – 先验

■ 全局刚性

$$\diamond E_{reg} = E_{match} + E_{prior}$$

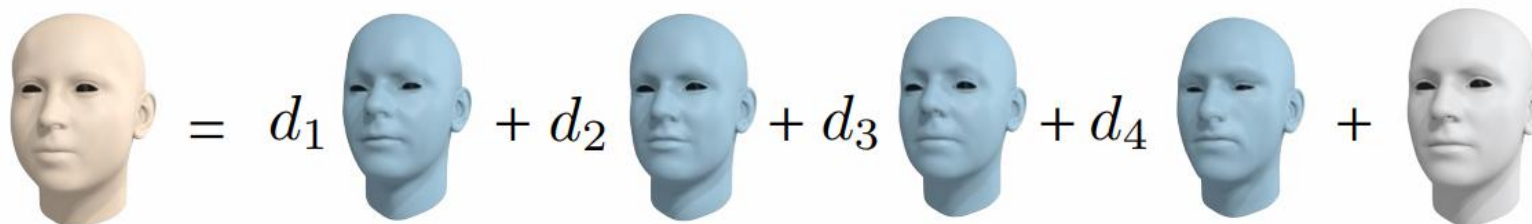


$$\diamond E_{prior}(Z, R, t) = \sum_{i=1}^n \|z_i - (Rx_i + t)\|_2^2$$

3D配准 – 先验

■ 线性模型

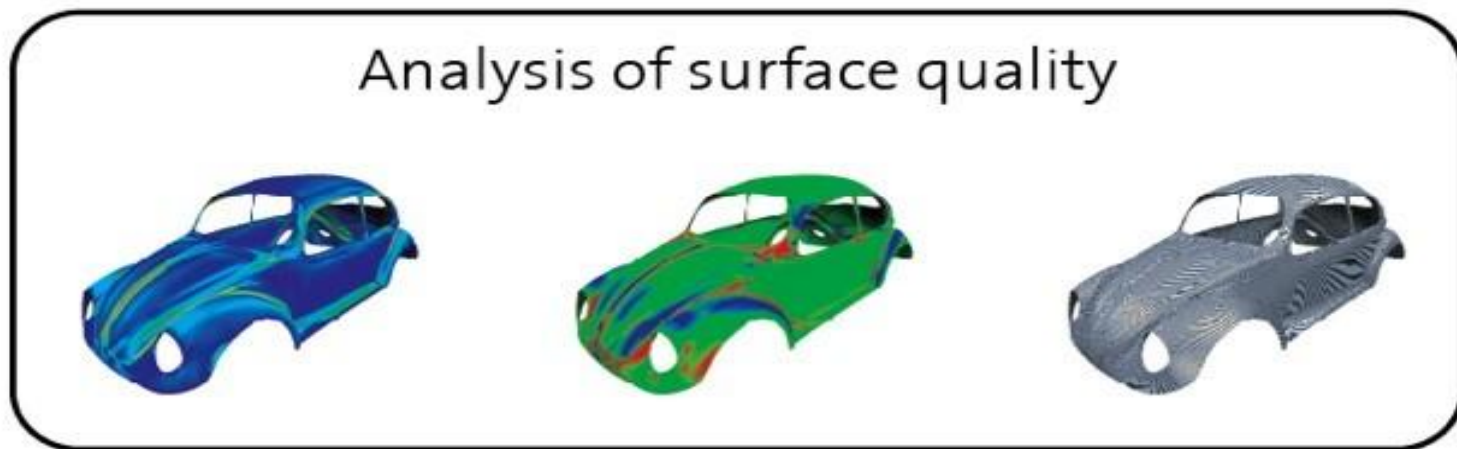
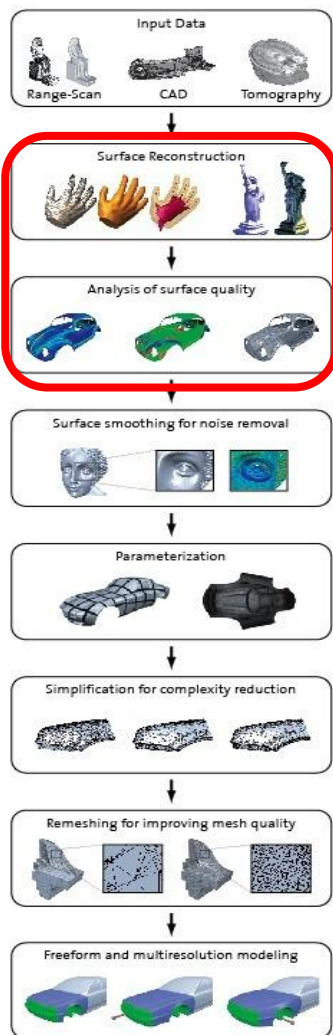
$$\diamond E_{reg} = E_{match} + E_{prior}$$



$$\diamond s = Pd + m$$

Mean shape

几何处理流程



网格相关基本概念

■ 网格的描述

- ◆ 一系列的面片 $F = (f_1, f_2, \dots, f_n)$, 每一个面片都是三角形
- ◆ 一系列的顶点 $V = (v_1, v_2, \dots, v_n)$
- ◆ F 中的每个面片是 V 中顶点的序列组 e.g.

$$f_1 = (v_1, v_2, v_3),$$

$$f_2 = (v_4, v_5, v_6),$$

$$f_3 = (v_7, v_8, v_9),$$

...

网格表示的由来

- 计算机产生的三维模型和实际获取的数据的表示模式不同
- 图形学中需要一个统一的表示方式
 - ◆ 视觉精度和处理速度需要在可以接受的范围之内
 - ◆ 由于图形硬件的快速发展, 我们已经能够快速地进行光栅化 (rasterize) 和三角网格的渲染 (render) 操作

三维数据的来源

- 模型产生途径:
- 直接在几何文件中输入
 - ◆ 程序建模 (procedure modeling) : 通过程序代码进行创建
 - ◆ 运用建模软件创建, 比如3ds/max, maya
 - ◆ 使用3D扫描仪 (3D Scanner) 获取真实模型
 - ◆ 照片测量法 (photogrammetry): 基于照片进行三维重构(Reconstruction)
 - ◆ 各类方法的组合

网格化(Mesh Tessellation)

■ 2D网格化

- ◆ 2D网格化在三维网格处理中有重要应用
- ◆ 多边形可以表示成多种不同的形式并且需要分割成更易于处理的图元，比如凸多边形，三角形或者四边形，这种表示过程称之为网格化(Mesh Tessellation)；其中，多边形分割成三角形的形式叫做三角化（triangulation）。

三角网格模型的存储

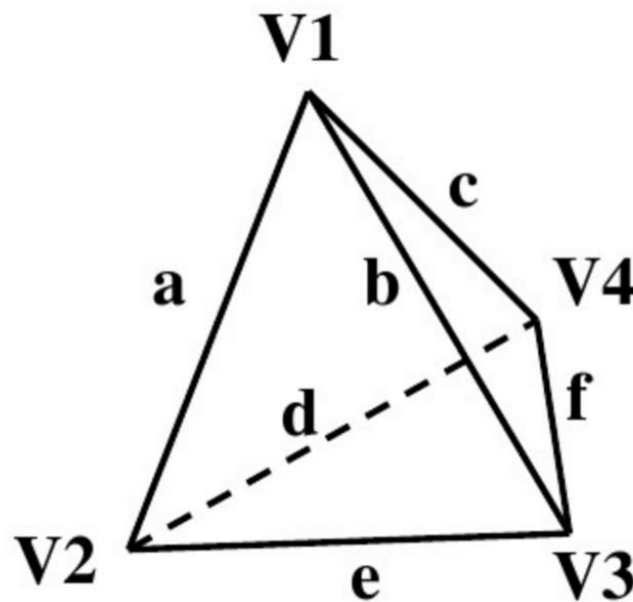
- 需要存储每个顶点的位置和所有的三角面片。
- OBJ文件格式的存储举例：

```
V1 x1 y1 z1  
V2 x2 y2 z2  
V3 x3 y3 z3  
V4 x4 y4 z4
```

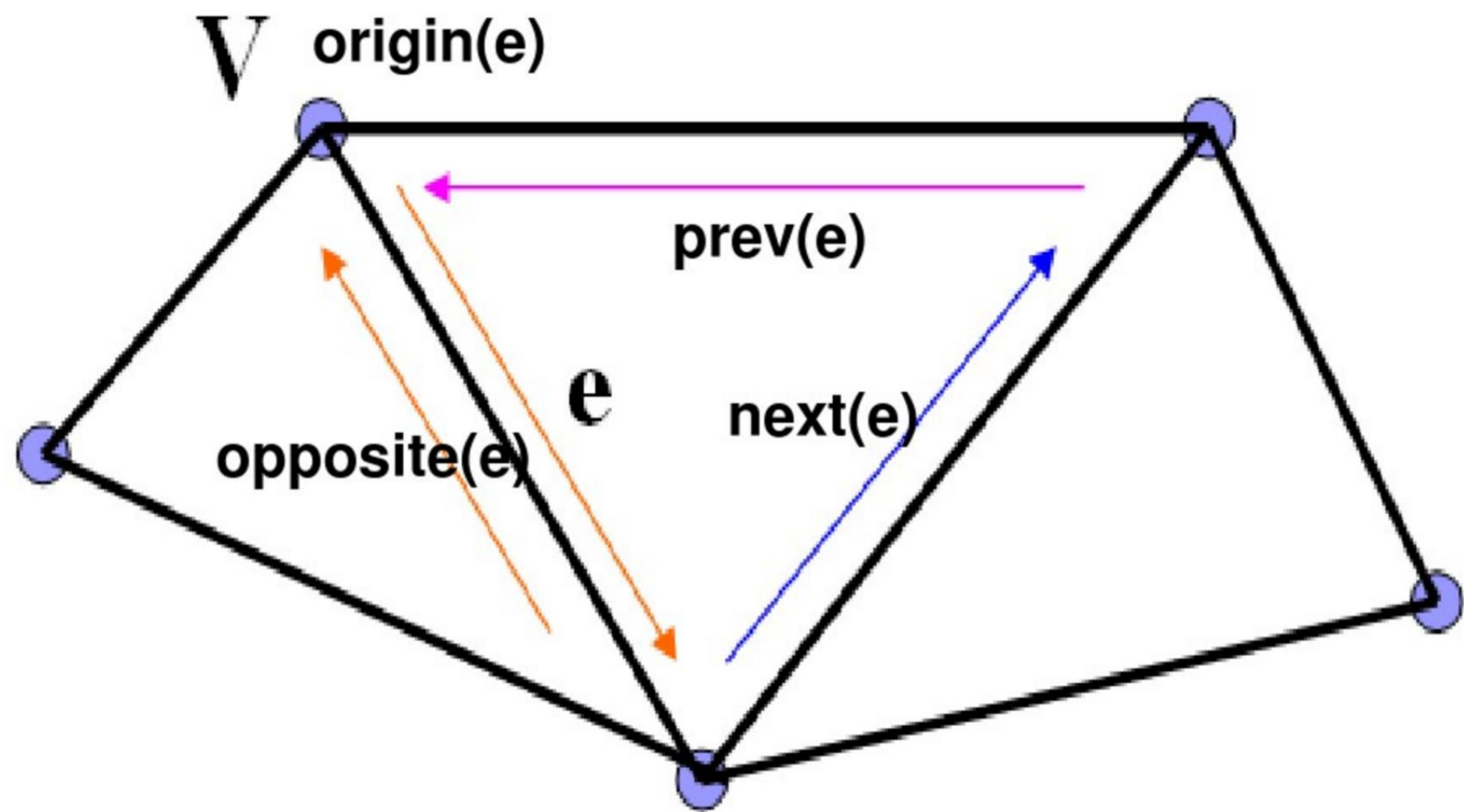
存顶点

```
f1 V1 V2 V3  
f2 V1 V3 V4  
f3 V1 V2 V4  
f4 V2 V3 V4
```

存面片



三角网格的半边表示

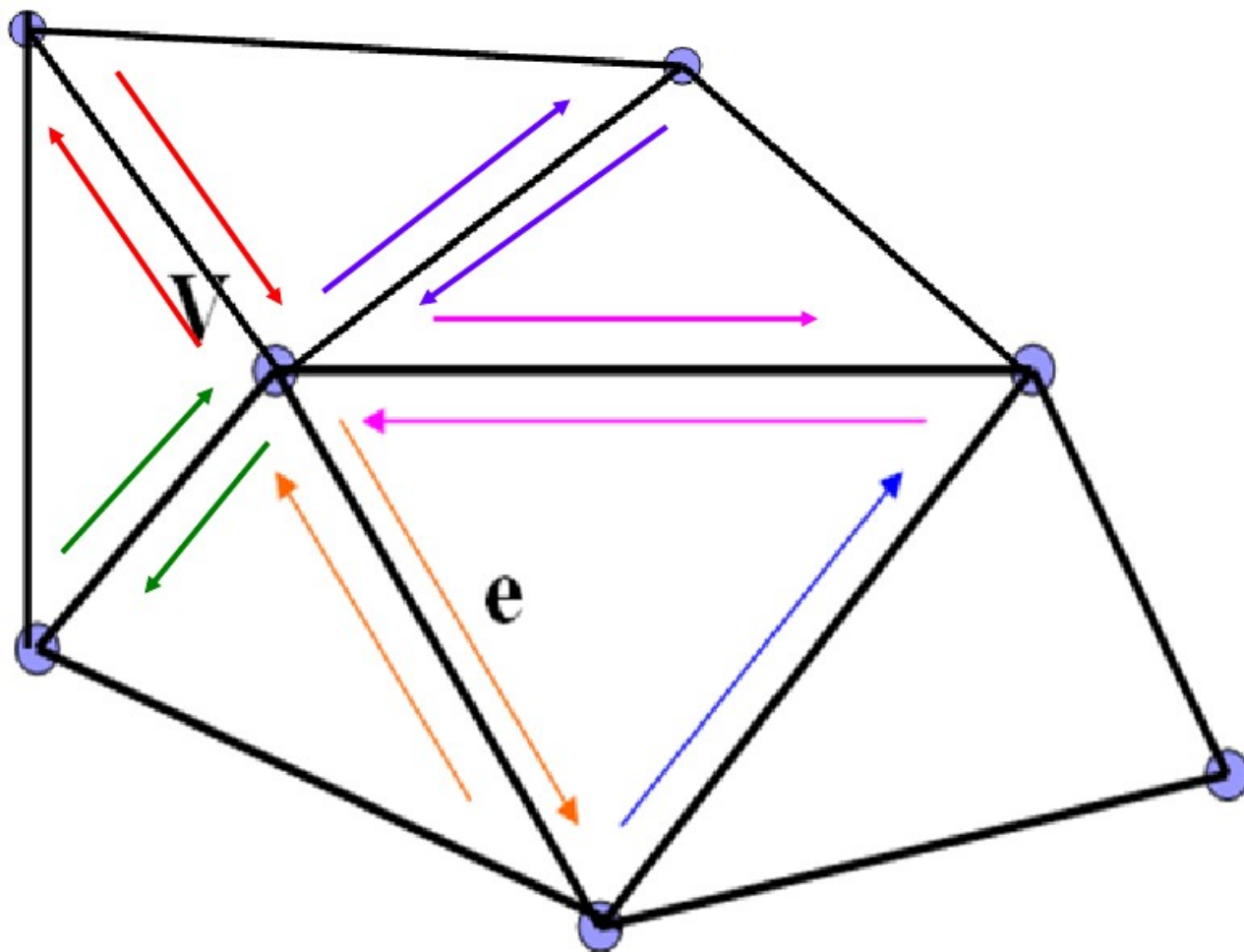


三角网格的半边表示

- 在半边结构中，每个顶点至少保存两部分信息：**顶点的集合信息**和**从该顶点出发的一条半边**。
- 在半边结构中，每条边需要存储的信息为：
 - ◆ 该半边的目标顶点 $\text{tar}(e)$;
 - ◆ 该半边在同一三角形的下一条半边 $\text{next}(e)$;
 - ◆ 该半边属于同一条边的对边 $\text{opposite}(e)$;
 - ◆ 该半边所属的面 $\text{IncFace}(e)$

三角网格的半边表示

- 按逆时针方向遍历包含某个顶点的所有面举例：



三角网格的半边表示

Struct HalfEdge

{

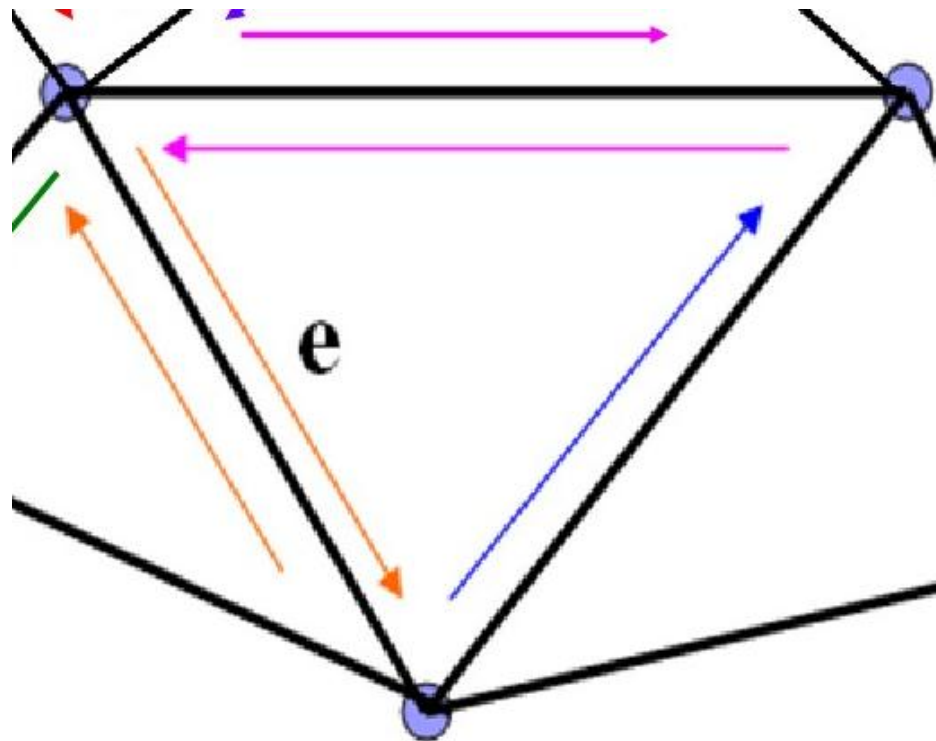
Vertex *vert;

HalfEdge *pair;

Face *face;

HalfEdge *next;

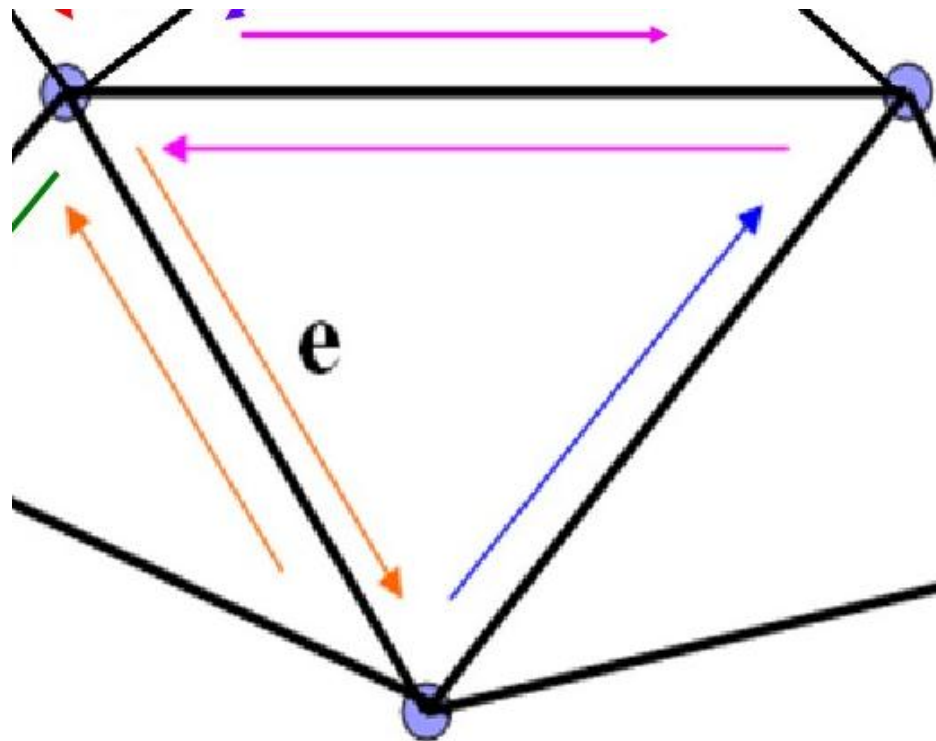
}



三角网格的半边表示

Struct Vertex

```
{  
float vcoord[3];  
float ncoord[3];  
HalfEdge *he;  
}
```



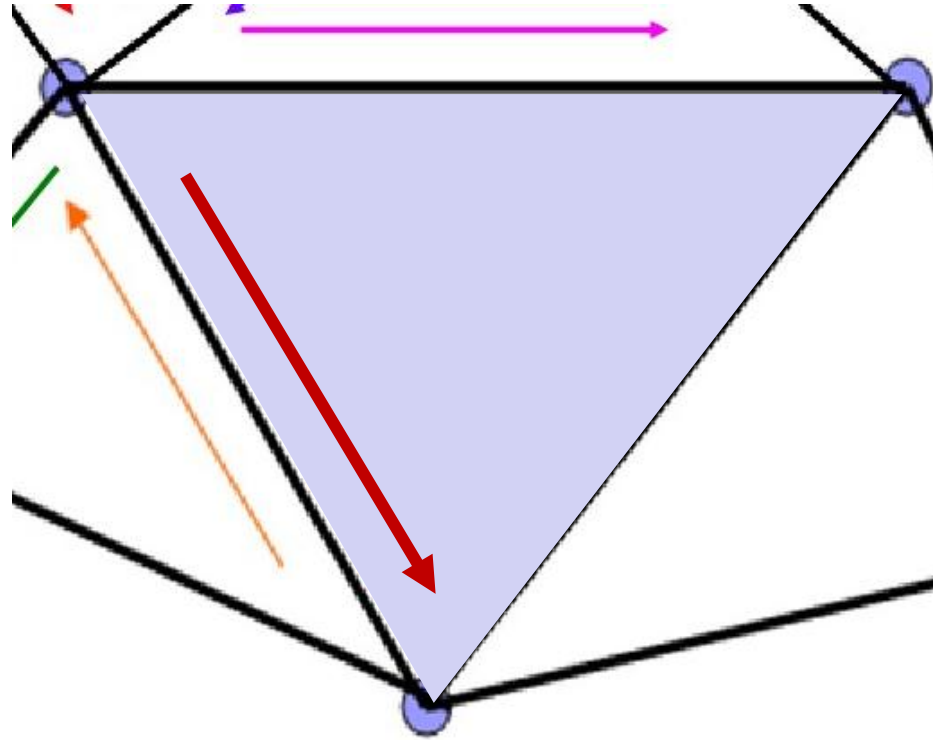
网格半边结构

Struct Face

{

HalfEdge *he;

}



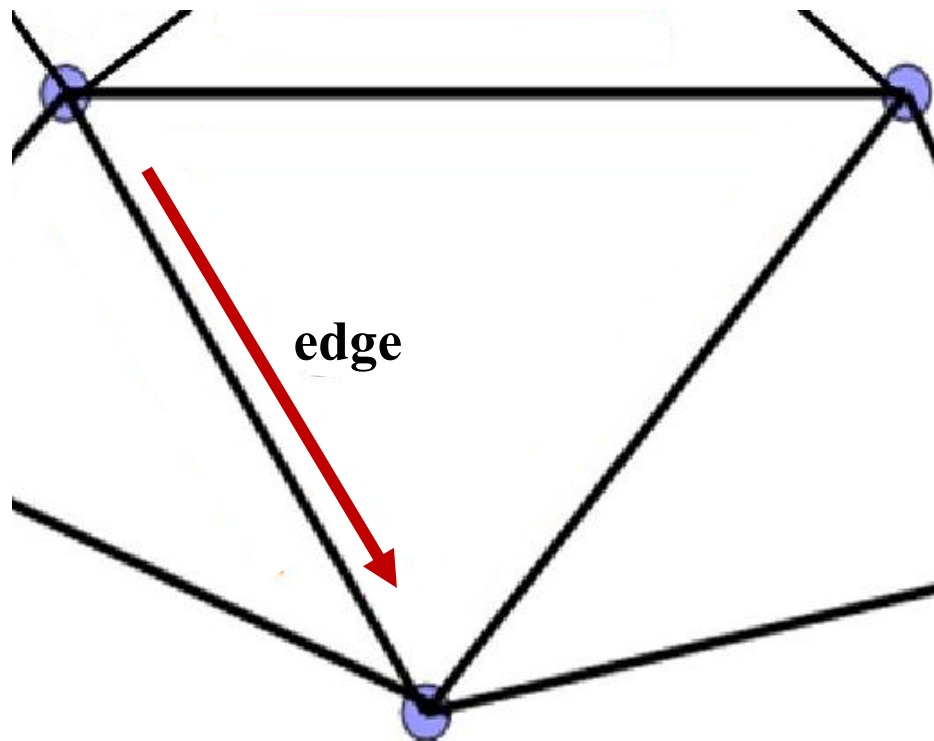
三角网格的半边表示

Vertex *vert1 = edge-> vert;

Vertex *vert2 = edge-> pair-> vert;

Face *face1 = edge-> face;

Face *face2 = edge-> pair-> face;



邻接查询

Vertex *vert1 = edge-> vert;

Vertex *vert2 = edge-> pair-> vert;

Face *face1 = edge-> face;

Face *face2 = edge-> pair-> face;

Struct HalfEdge

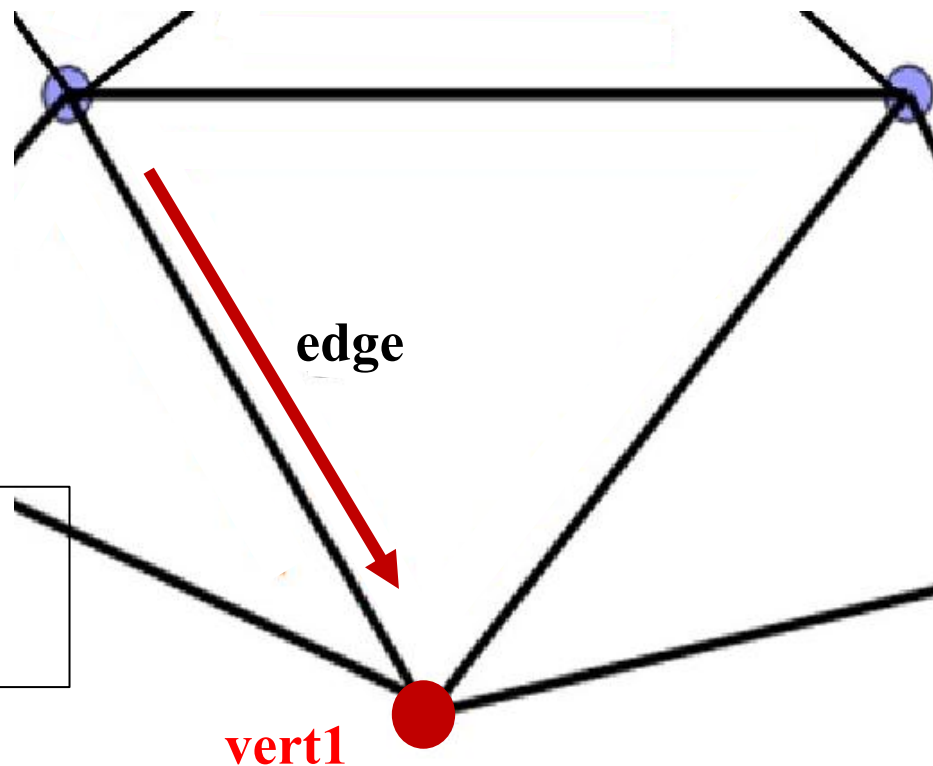
```
{  
    Vertex *vert;  
    HalfEdge *pre;  
    HalfEdge *next;  
    Face *face;  
}
```

Struct Vertex

```
{  
    float vcoord[3];  
    float ncoord[3];  
    HalfEdge *he;  
}
```

Struct Face

```
{  
    HalfEdge *he;  
}
```



邻接查询

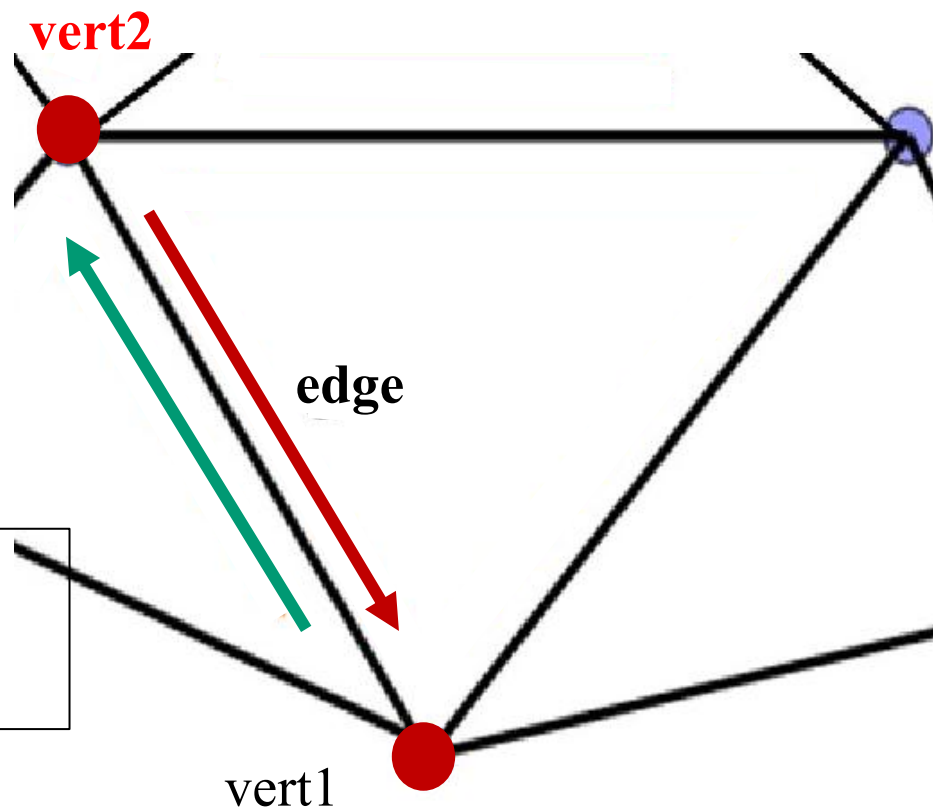
Vertex *vert1 = edge-> vert;

Vertex *vert2 = edge-> pair-> vert;

Face *face1 = edge-> face;

Face *face2 = edge-> pair-> face;

Struct HalfEdge	Struct Vertex	Struct Face
{ Vertex *vert; HalfEdge *pre; HalfEdge *next; Face *face; }	{ float vcoord[3]; float ncoord[3]; HalfEdge *he; }	{ HalfEdge *he; }



邻接查询

```
Vertex *vert1 = edge-> vert;
```

```
Vertex *vert2 = edge-> pair-> vert;
```

```
Face *face1 = edge-> face;
```

```
Face *face2 = edge-> pair-> face;
```

Struct HalfEdge

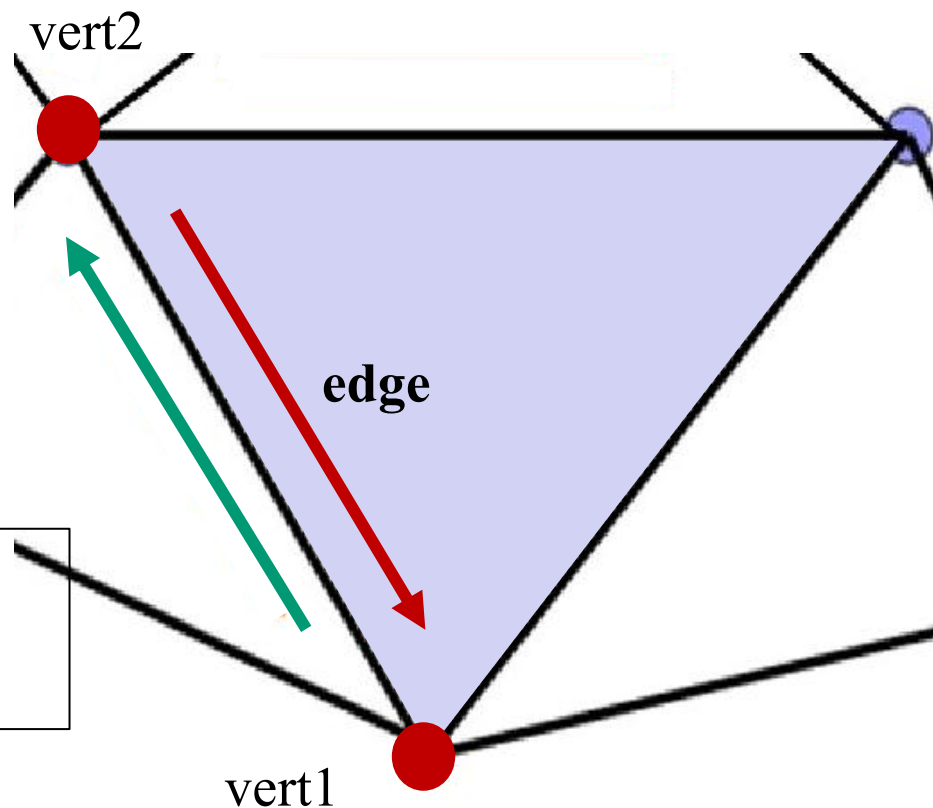
```
{  
    Vertex *vert;  
    HalfEdge *pre;  
    HalfEdge *next;  
    Face *face;  
}
```

Struct Vertex

```
{  
    float vcoord[3];  
    float ncoord[3];  
    HalfEdge *he;  
}
```

Struct Face

```
{  
    HalfEdge *he;  
}
```



邻接查询

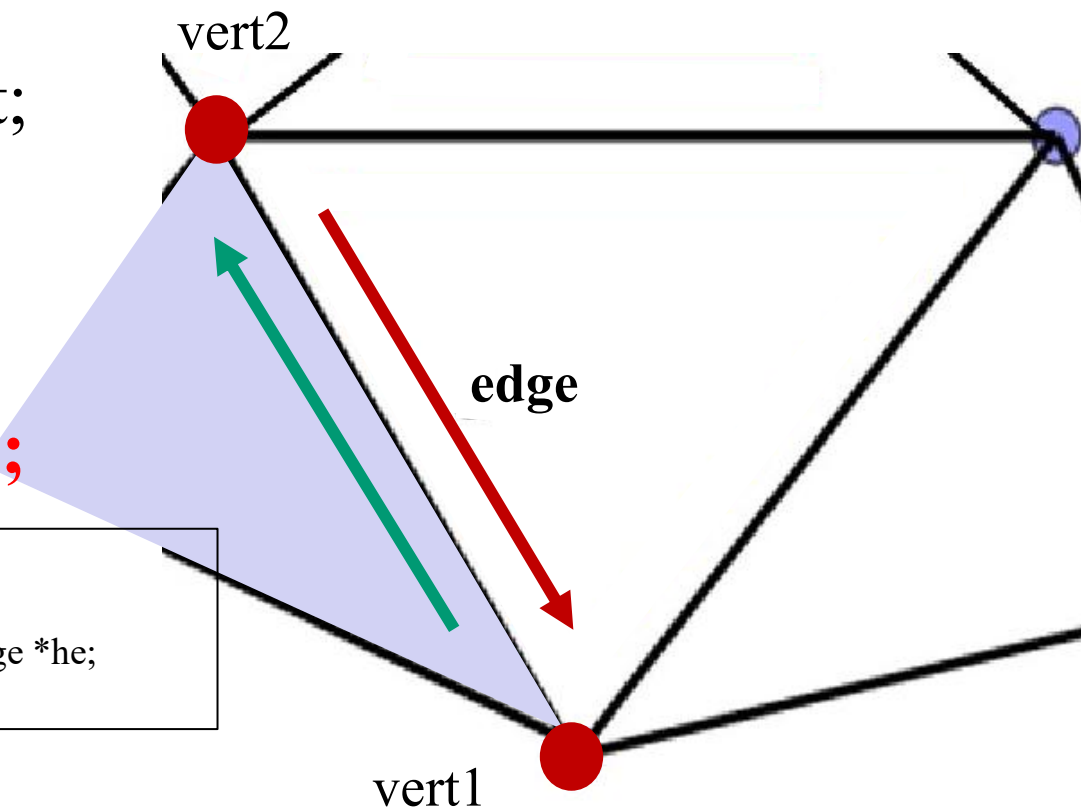
Vertex *vert1 = edge-> vert;

Vertex *vert2 = edge-> pair-> vert;

Face *face1 = edge-> face;

Face *face2 = edge-> pair-> face;

Struct HalfEdge	Struct Vertex	Struct Face
{ Vertex *vert; HalfEdge *pre; HalfEdge *next; Face *face; }	{ float vcoord[3]; float ncoord[3]; HalfEdge *he; }	{ HalfEdge *he; }



邻接查询

- 如何遍历一个面片上的所有半边结构？

```
HalfEdge *edge = face->edge;
```

```
do{
```

```
    edge = edge->next;
```

```
}while (edge != face->edge);
```

邻接查询

■ 如何遍历一个顶点周围的半边？

```
HalfEdge *edge = vert->edge;
```

```
do{
```

```
    edge = edge->pair->next;
```

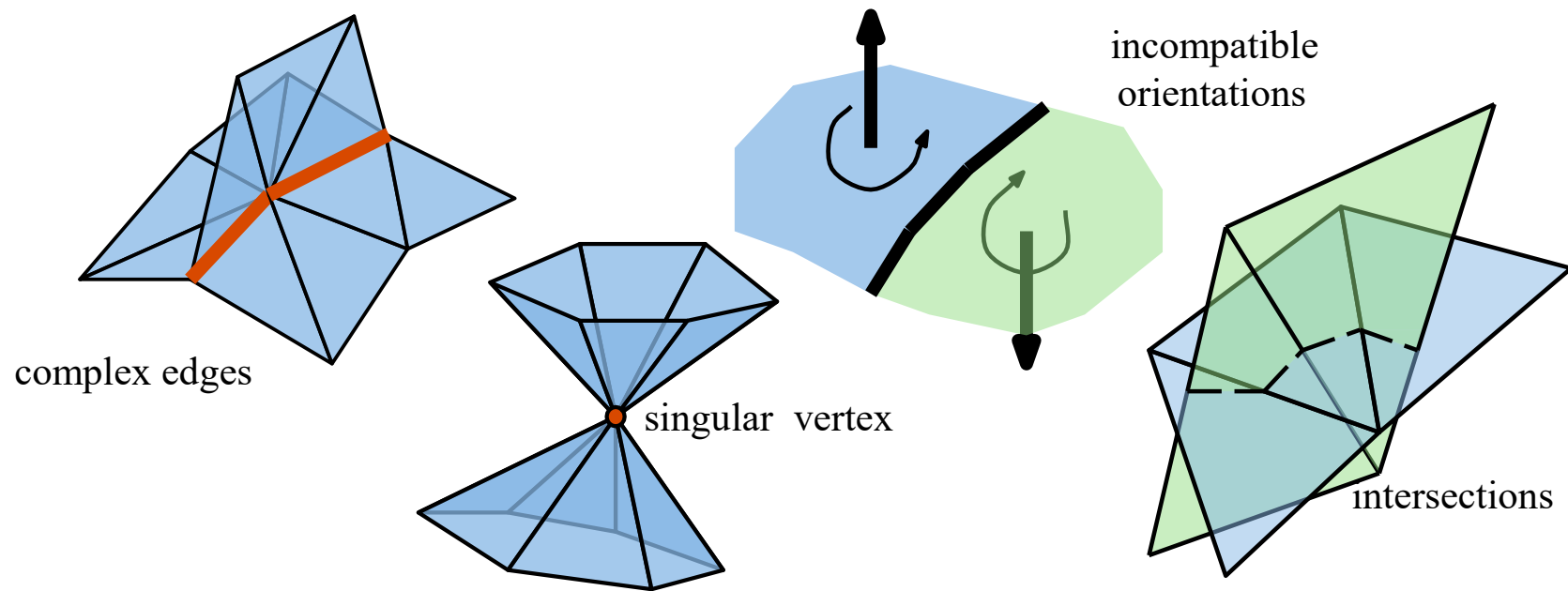
```
}while (edge != vert->edge);
```

邻接查询

- 如何遍历使用这个顶点的面片？
- 如何查询边所在的面片？
- 如何查询与某面片邻接的面片？

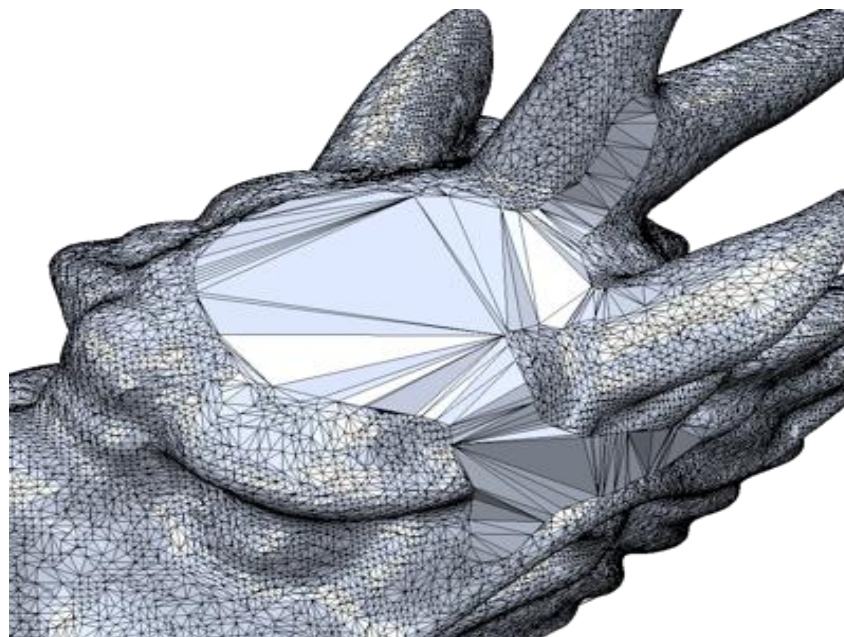
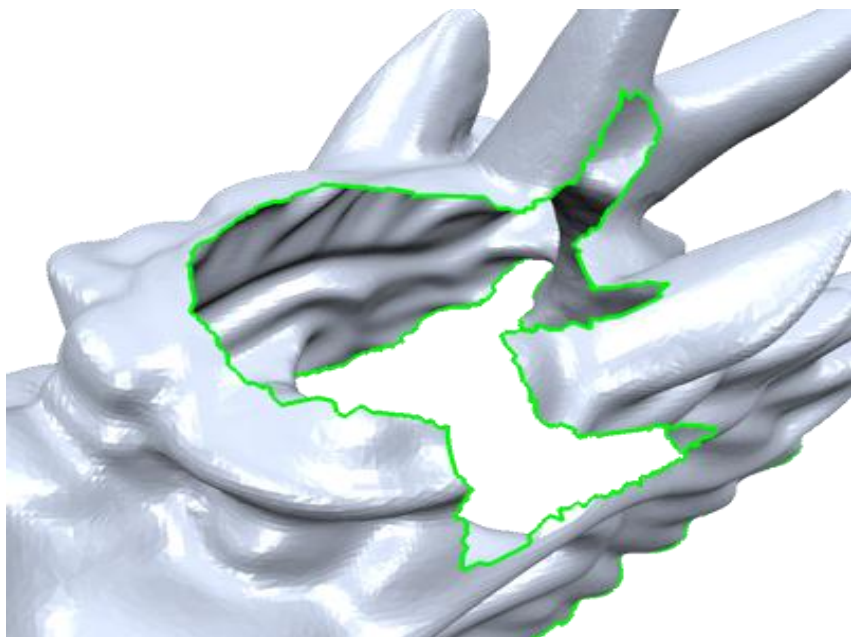
Triangle Soups

- 有利于可视化，但不利于需要流形网格的下游应用
- 以及我们已经遇见的模型：



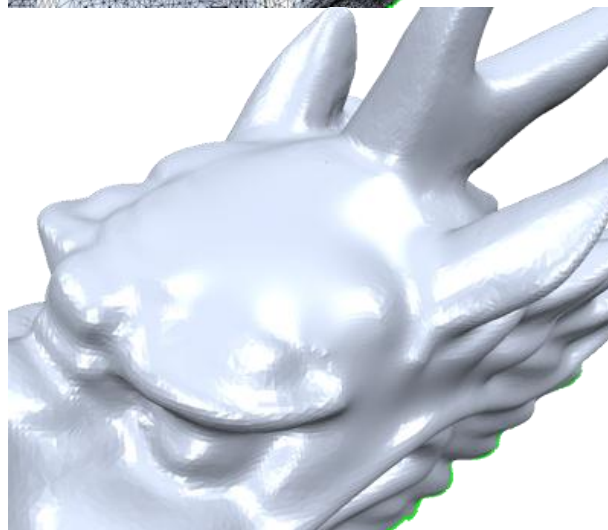
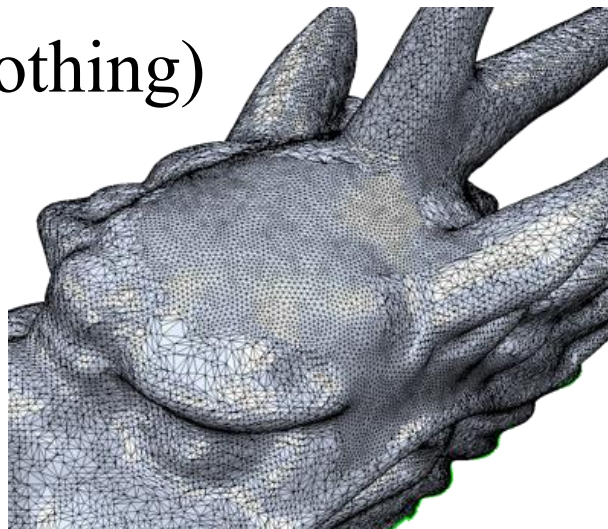
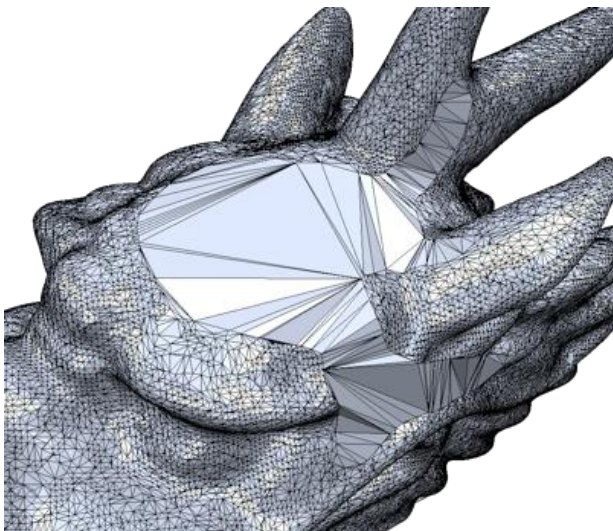
填充孔洞

- 计算一个粗糙的三角化T(Compute a coarse triangulation T)



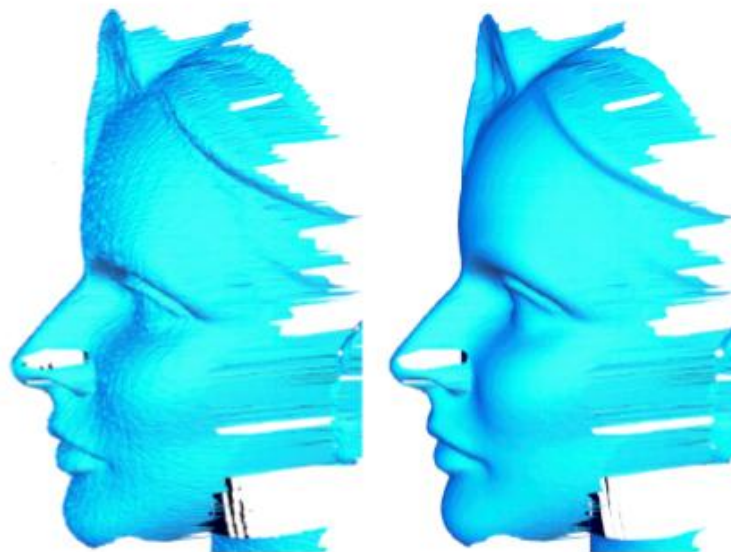
填充孔洞

- 优化和平滑(Refinement and smoothing)



Outline

- 拉普拉斯平滑



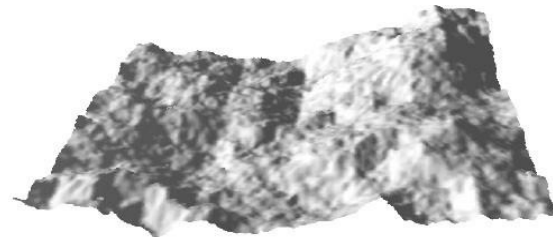
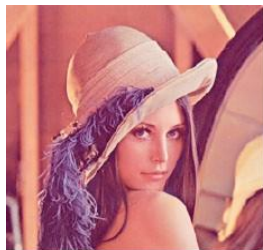
Diffusion

■ Diffusion 方程

diffusion 常量

$$\frac{\partial}{\partial t} x = \mu \Delta x$$

Laplace 算子



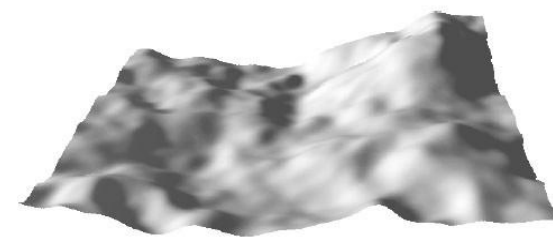
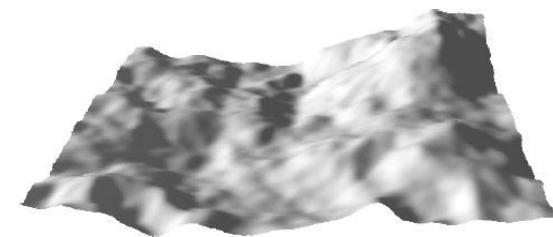
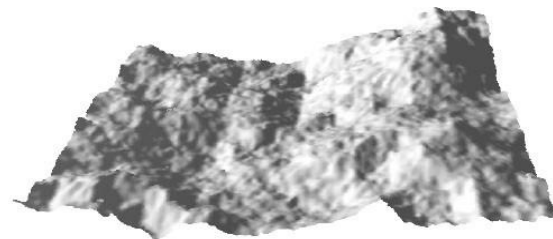
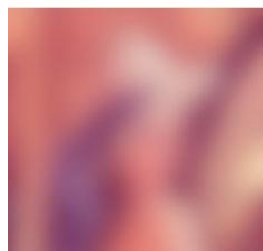
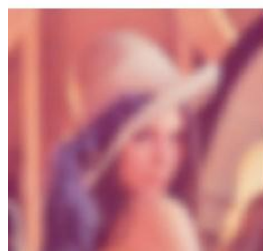
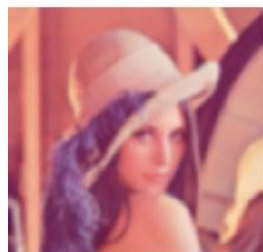
Diffusion

■ Diffusion方程

diffusion 常量

$$\frac{\partial}{\partial t} x = \mu \Delta x$$

Laplace算子



拉普拉斯平滑

■ diffusion方程的离散化

$$\frac{\partial}{\partial t} p_i = \mu \Delta p_i$$

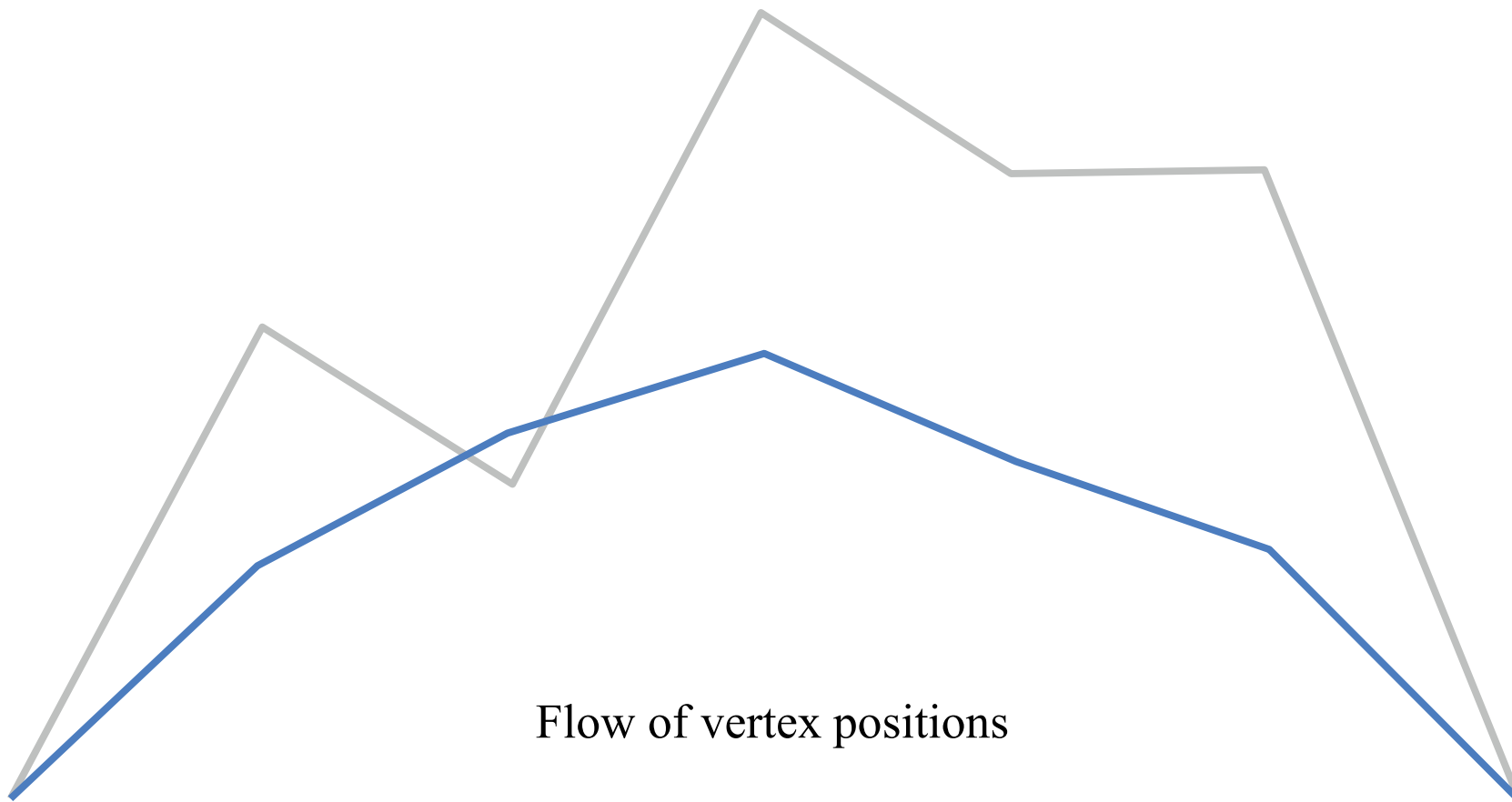
■ 更新

◆ 迭代

$$p'_i = p_i + \mu dt \Delta p_i$$

◆ 直到收敛

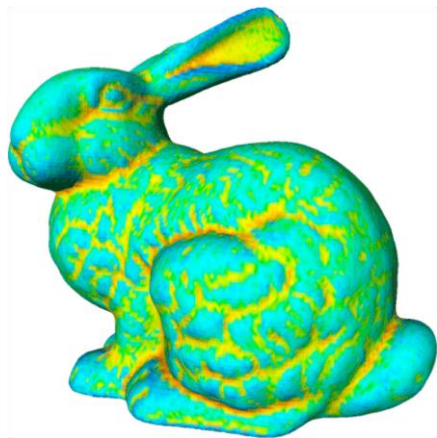
简单示例



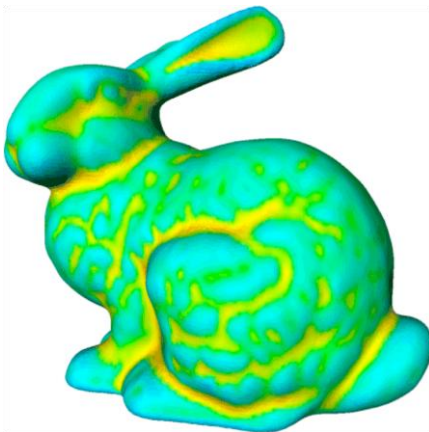
简单示例



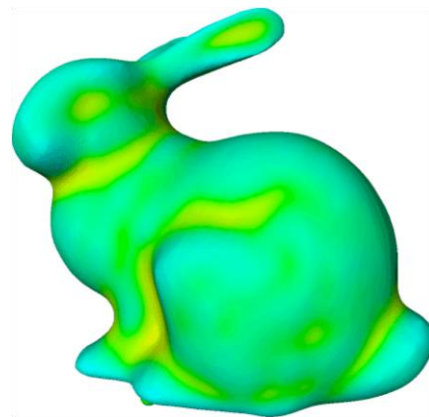
拉普拉斯平滑



0 Iterations

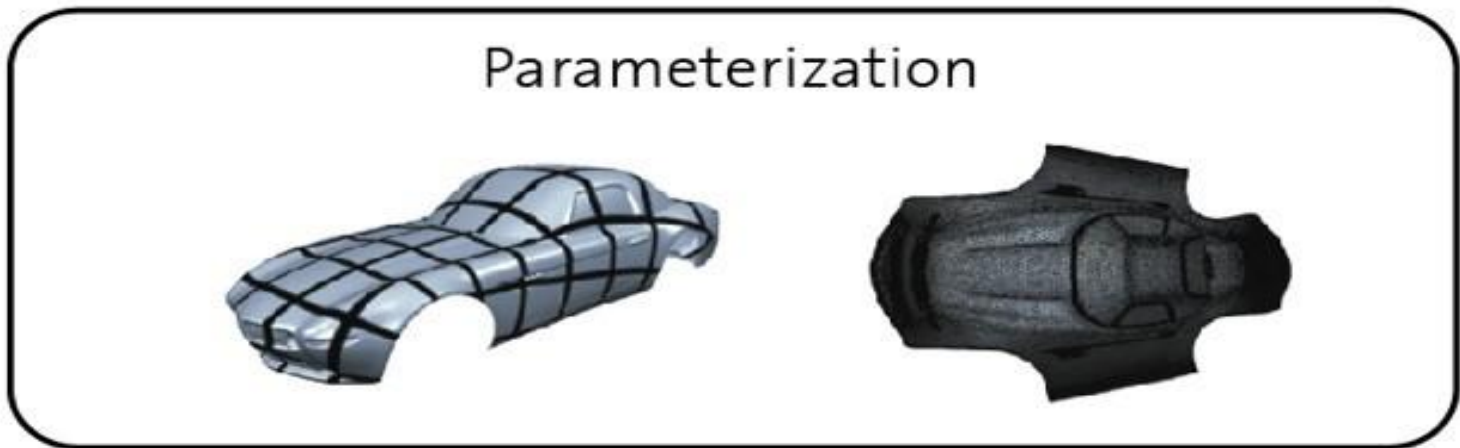
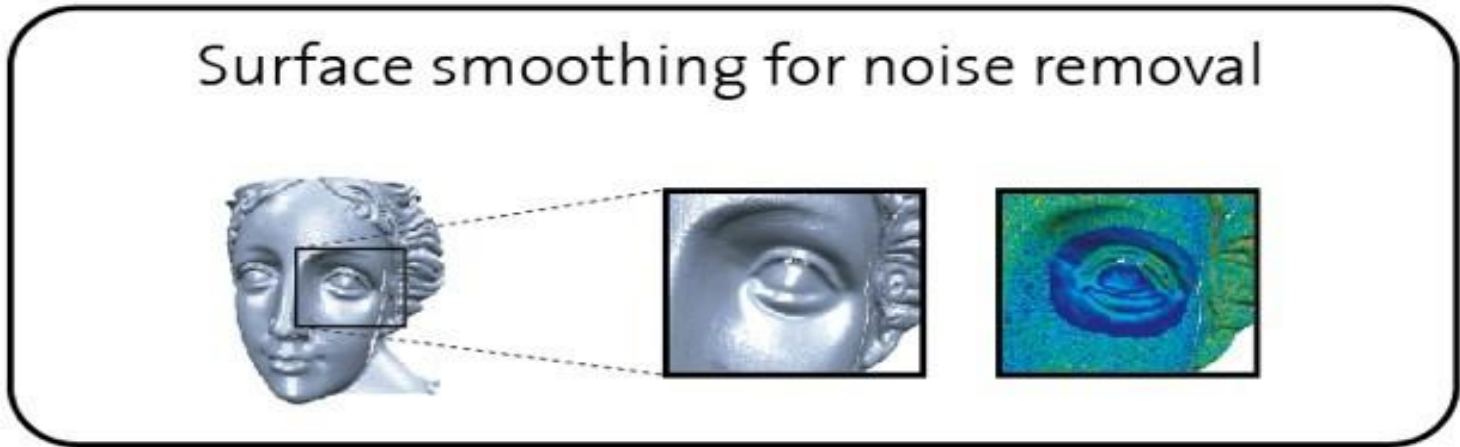
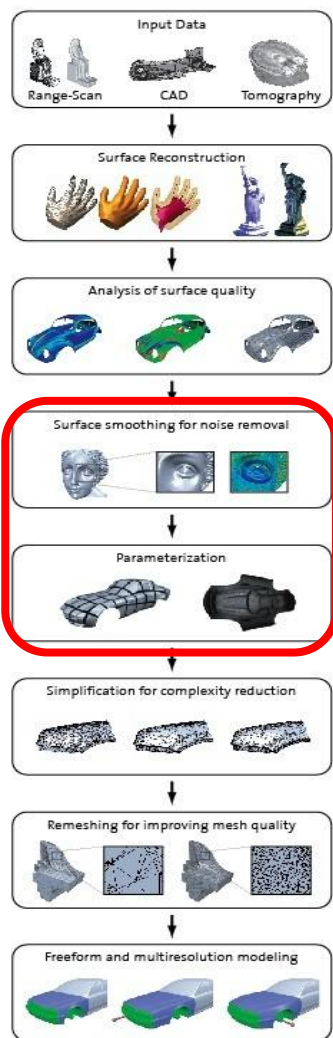


5 Iterations



20 Iterations

几何处理流程



参数化

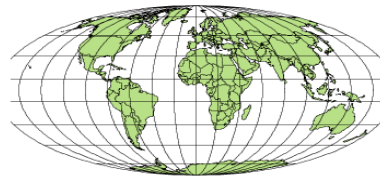


Mercator-Projektion



Mollweide-Projektion

参数化



Mollweide-Projektion



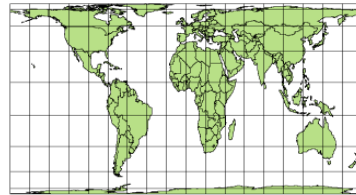
Mercator-Projektion



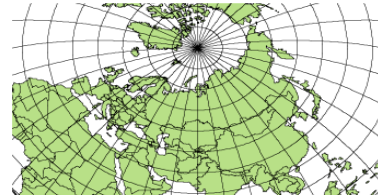
Zylinderprojektion nach Miller



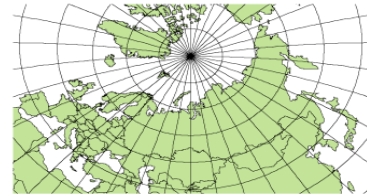
Hammer-Aitoff-Projektion



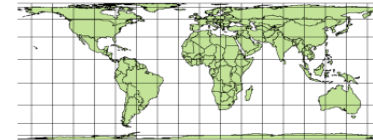
Peters-Projektion



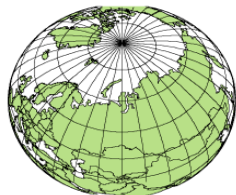
Längentreue Azimuthalprojektion



Stereographische Projektion



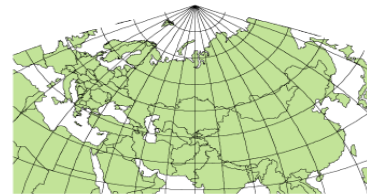
Behrmann-Projektion



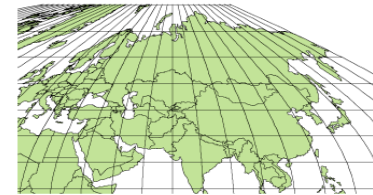
Senkrechte Umgebungsperspektive



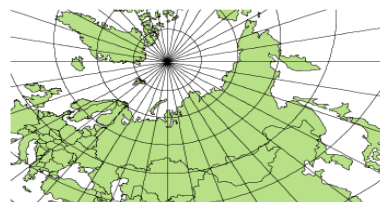
Robinson-Projektion



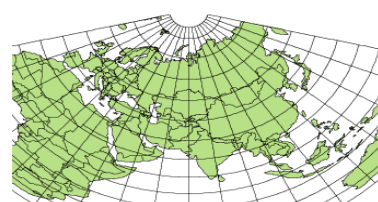
Hotine Oblique Mercator-Projektion



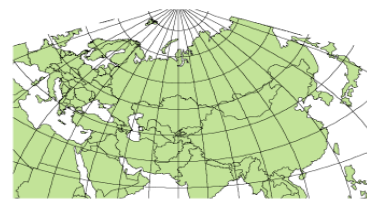
Sinusoidale Projektion



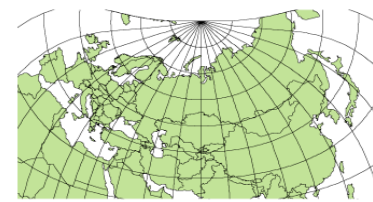
Gnomonische Projektion



Flächentreue Kegelprojektion

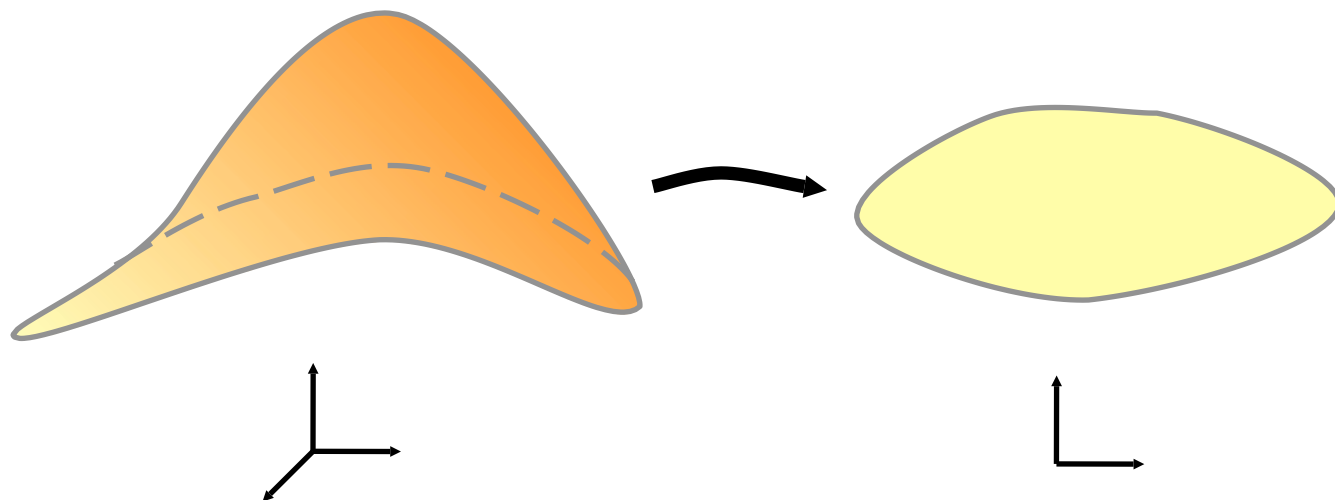


Transverse Mercator-Projektion



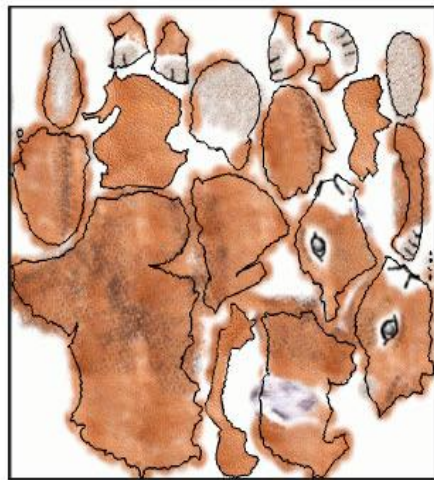
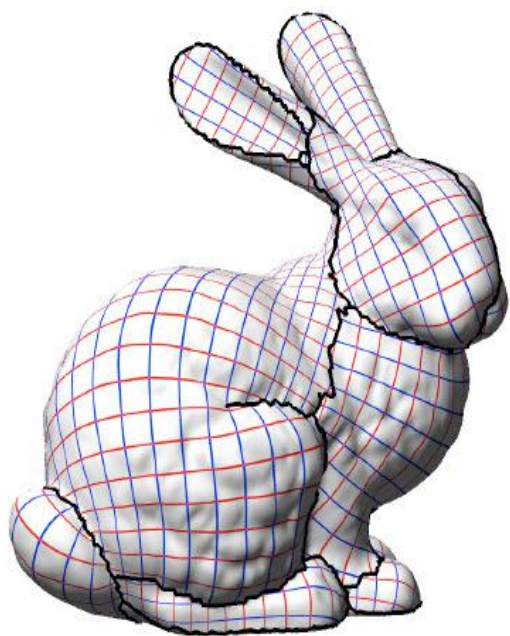
Cassini-Soldner-Projektion

参数化



动机

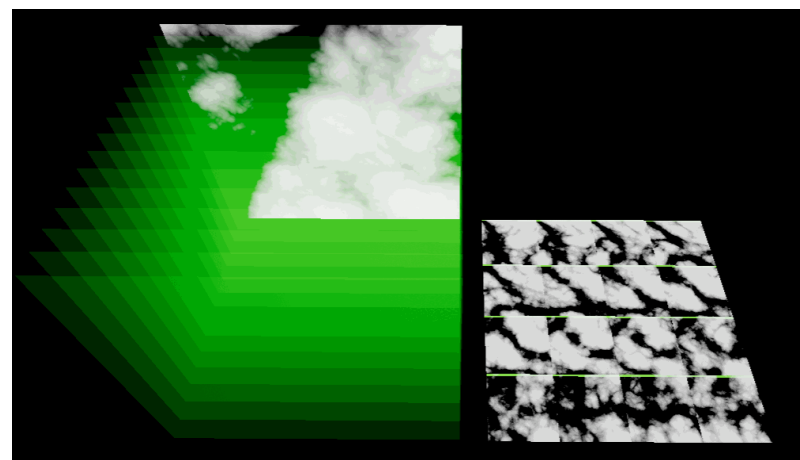
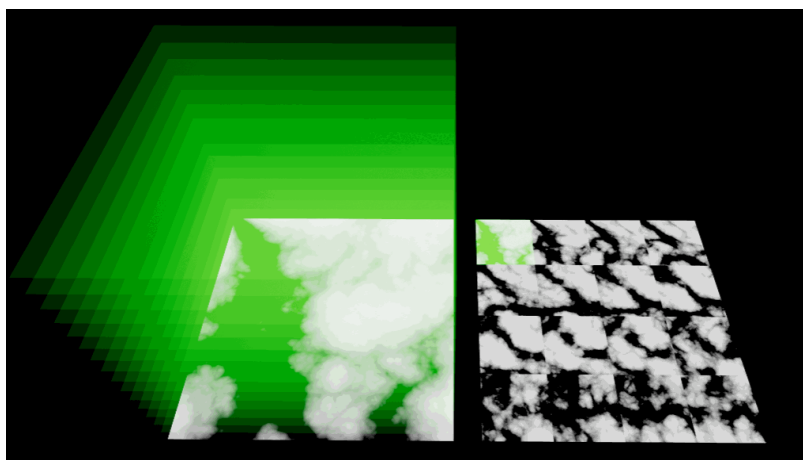
■ 纹理映射



Lévy, Petitjean, Ray, and Maillot: *Least squares conformal maps for automatic texture atlas generation*, SIGGRAPH 2002

体纹理

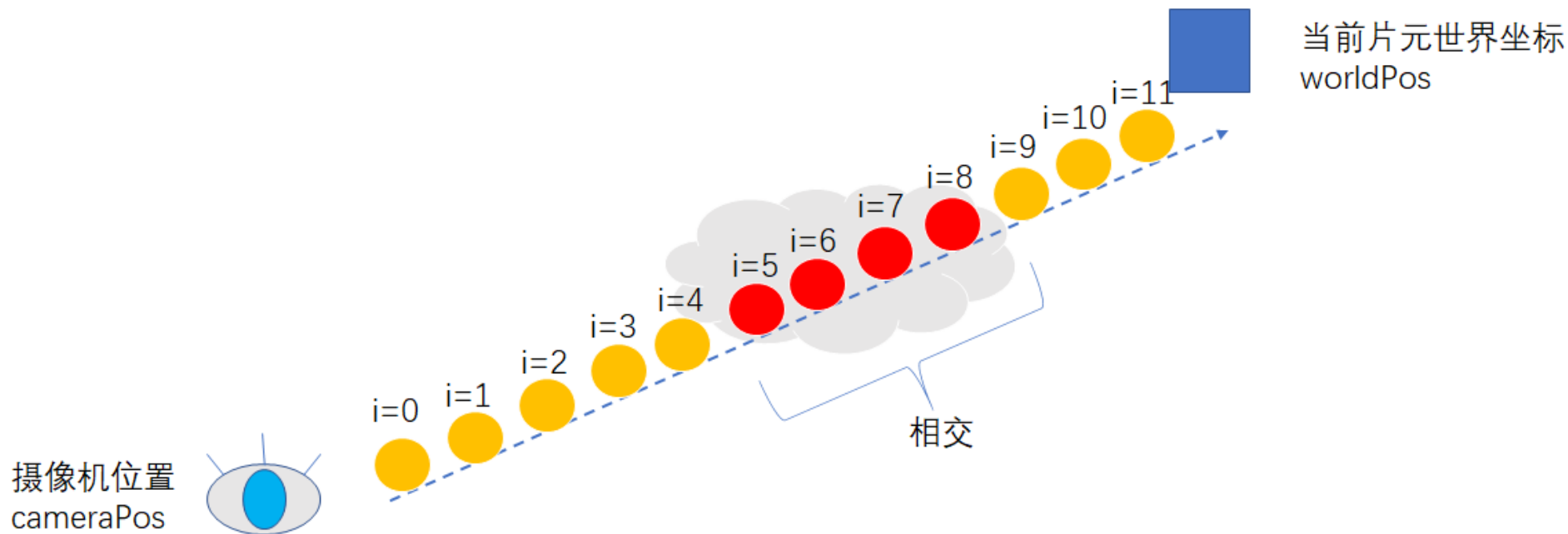
- 纹理值构成的三维数组(例如：一个大理石块)。
- 利用一个 $[xyz] \rightarrow [RGB]$ 的函数把颜色映射到点上。
- 这种三维纹理被称为体纹理。
- 实际使用时，这种映射通常用过程定义，不需要存储整个三维颜色数组，仅需定义一个为每个三维点产生颜色的函数。



体渲染

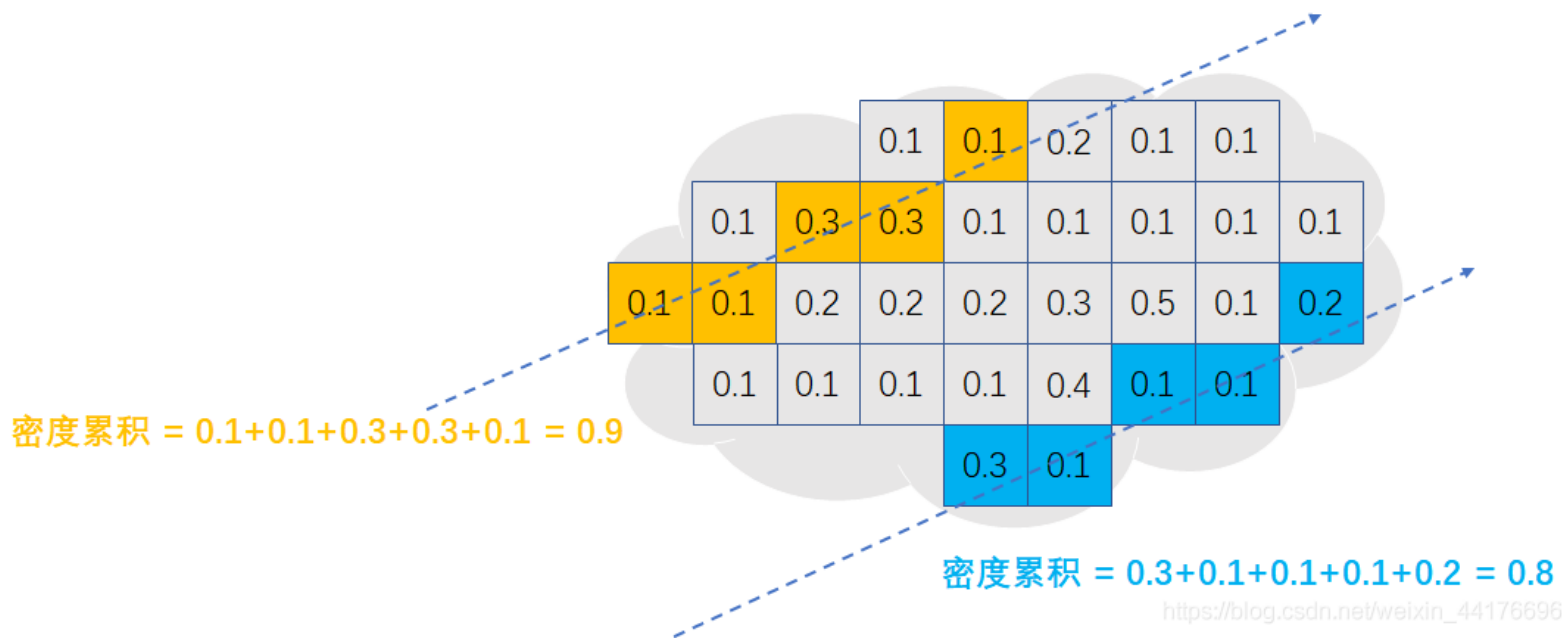
■ Ray marching 算法:

- ◆ 算法从摄像机开始，向世界空间投射光线，并且逐步行进，记录沿途的信息。
- ◆ 比如我们沿途不断判断当前点是否在云层中，如果沿途至少有一点在云层中，我们认为视线和云层相交。下图描述了 ray marching 算法的步进过程：



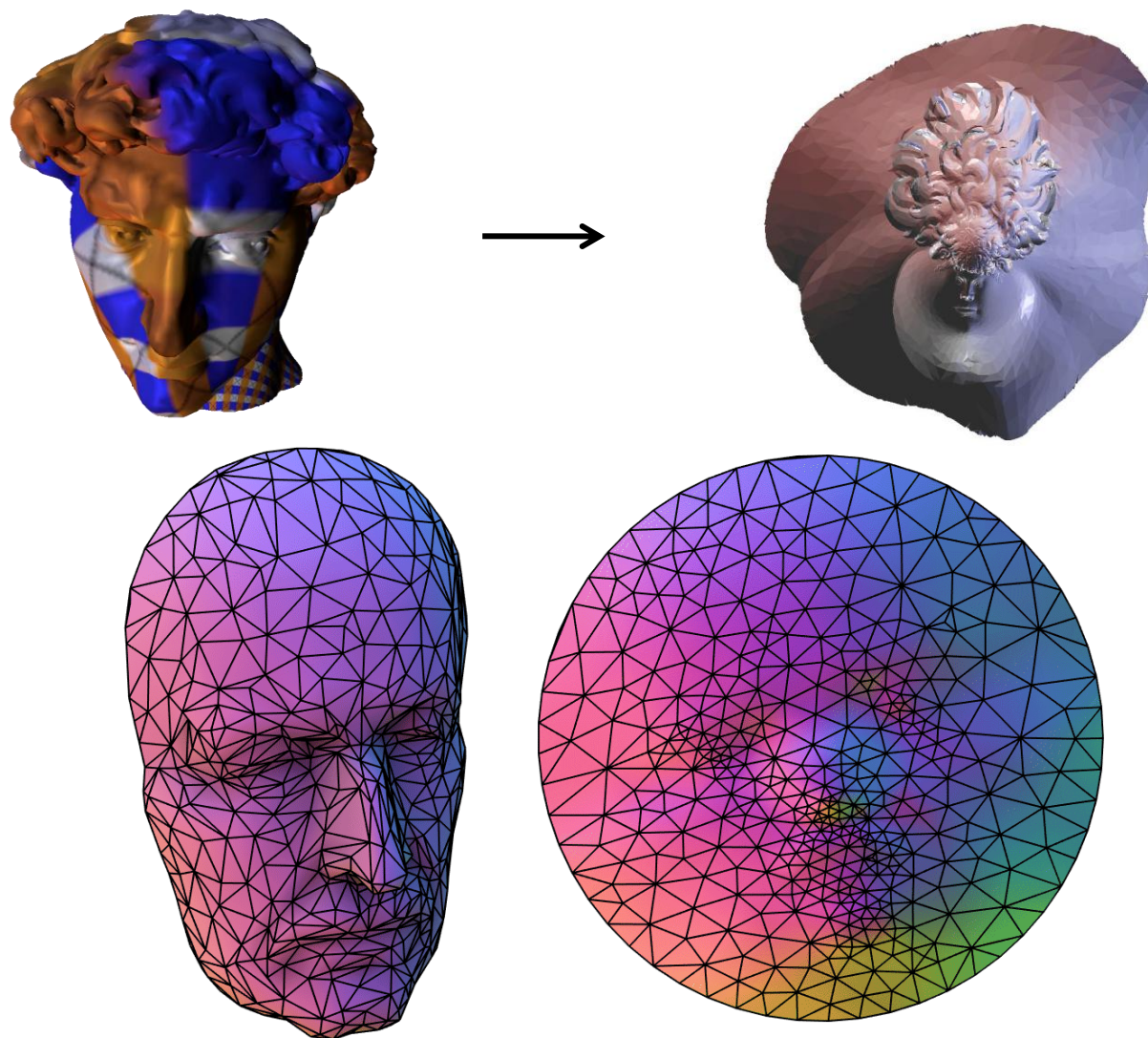
体渲染

- 以上的思路是针对具有明确边界的【固体】进行的，但是云朵通常是用一种没有具体边界的【密度函数】来描述的。密度函数的输入是三维的坐标，输出是当前坐标的云朵的密度。于是每次采样我们累积云朵的密度，就可以知道当前光线穿越了多厚的云。二维下的算法图示如下：

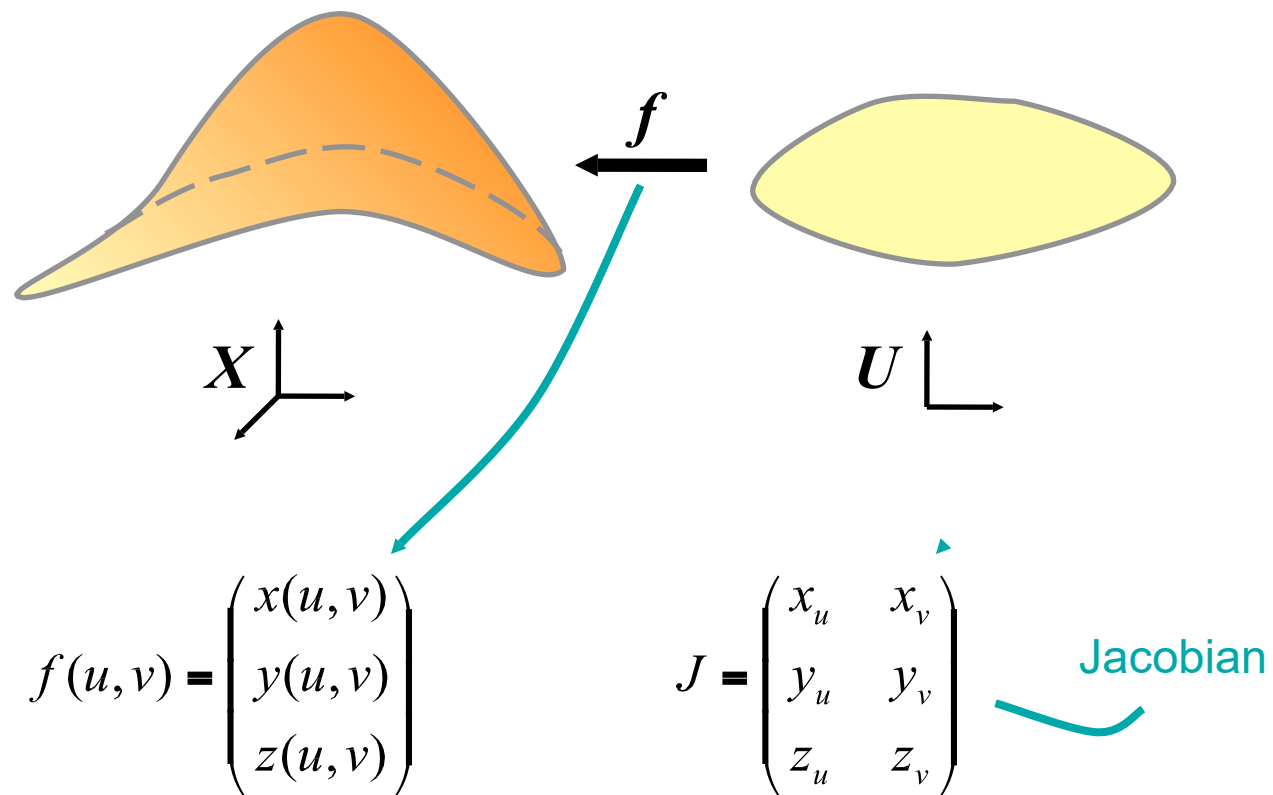


参数化

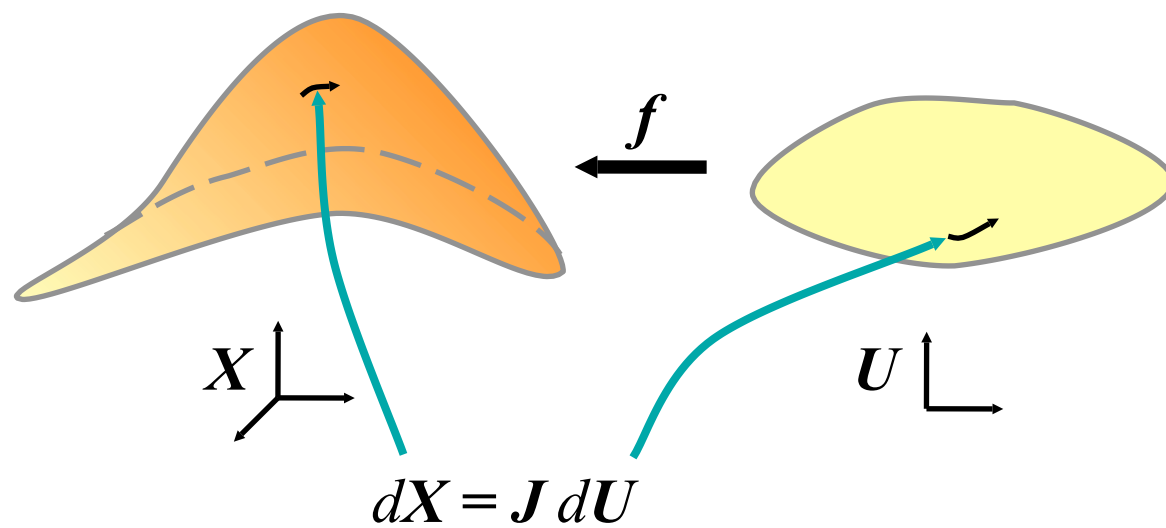
■ 纹理映射



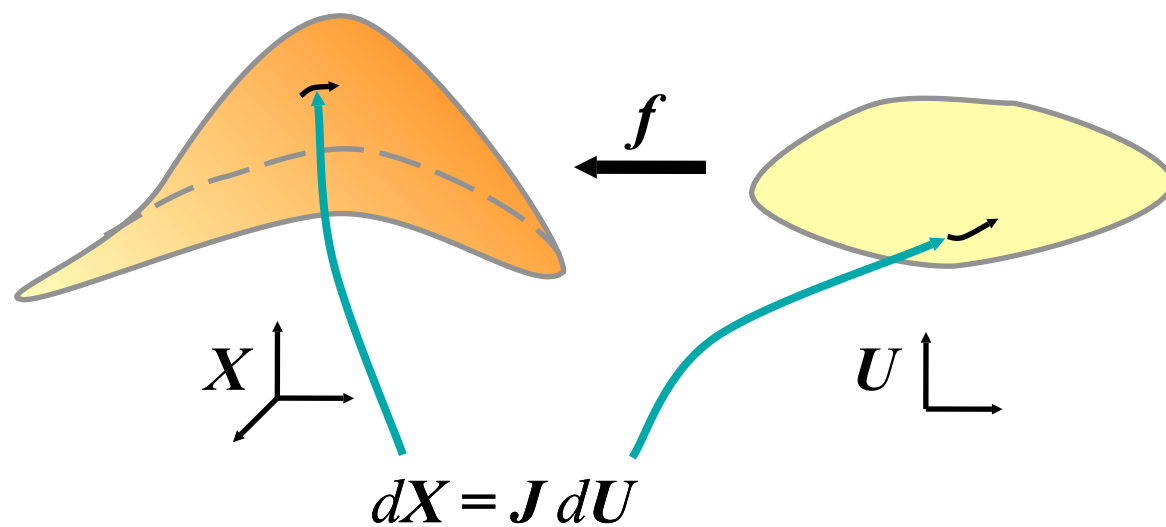
参数化



参数化



参数化

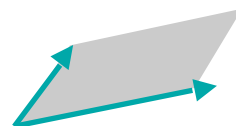
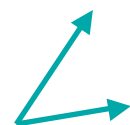
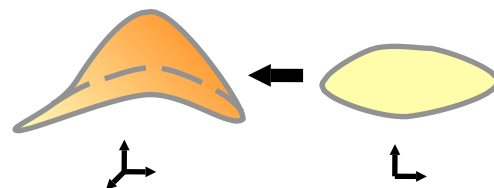


$$\|dX\|^2 = dU \underbrace{J^T J}_{\text{First Fundamental Form}} dU$$

$$\mathbf{I} = \begin{pmatrix} x_u x_u & x_u x_v \\ x_u x_v & x_v x_v \end{pmatrix}$$

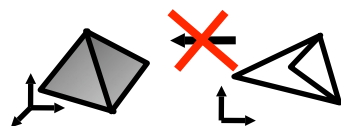
映射的特点

- 利用曲面第一基本型 I
 - I 的特征值 $\lambda_{1,2}$
 - I 的奇异值 $\sigma_{1,2}$ ($\sigma_{i,2} = \lambda_i$)
- 等距 (Isometric)
 - $I = Id$, $\lambda_1 = \lambda_2 = 1$
- 保角 (Conformal)
 - $I = \mu Id$, $\lambda_1 / \lambda_2 = 1$
- 保面积 (Equiareal)
 - $\Delta I = 1$, $\lambda_1 \lambda_2 = 1$

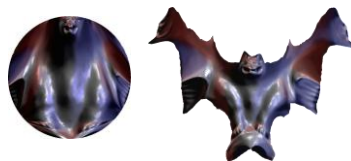


目标

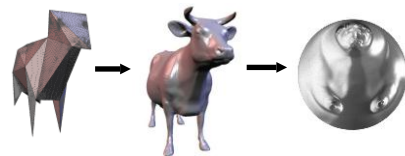
- 等距映射较少
- 最小化关于某一度量的形变
 - ◆ 有效性 (Validity) (双射)
 - ◆ 边界 (Boundary)
 - ◆ 域 (Domain)
 - ◆ 数值解 (Numerical solution)



triangle flip



fixed / free?



e.g., spherical

linear / non-linear?

谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn